FLEXIBLE STATISTICAL MODELING

A DISSERTATION

SUBMITTED TO THE DEPARTMENT OF STATISTICS

AND THE COMMITTEE ON GRADUATE STUDIES

OF STANFORD UNIVERSITY

IN PARTIAL FULFILLMENT OF THE REQUIREMENTS

FOR THE DEGREE OF

DOCTOR OF PHILOSOPHY

Ji Zhu

August 2003

I certify that I have read this dissertation and that, in my opinion, it is fully adequate in scope and quality as a dissertation for the degree of Doctor of Philosophy.

Trevor Hastie
(Principal Advisor)

I certify that I have read this dissertation and that, in my opinion, it is fully adequate in scope and quality as a dissertation for the degree of Doctor of Philosophy.

Bradley Efron

I certify that I have read this dissertation and that, in my opinion, it is fully adequate in scope and quality as a dissertation for the degree of Doctor of Philosophy.

Robert Tibshirani

Approved for the University Committee on Graduate Studies:

Abstract

The support vector machine is known for its good performance in two-class classification. The topic of this thesis is based on two variations of the standard 2-norm support vector machine.

In the first part of the thesis, we replace the *hinge* loss of the support vector machine with the negative binomial log-likelihood and consider the kernel logistic regression model. We show that kernel logistic regression performs as well as the support vector machine in two-class classification. Further more, kernel logistic regression provides an estimate of the underlying probability. Based on the kernel logistic regression model, we propose a new approach for classification, called the *import vector machine*. Similar to the *support points* of the support vector machine, the import vector machine model uses only a fraction of the training data to index kernel basis functions, typically a much smaller fraction than the support vector machine. This gives the import vector machine a computational advantage over the support vector machine, especially when the size of the training data set is large.

In the second part, we replace the L_2-norm penalty term of the support vector machine with the L_1-norm penalty, and consider the 1-norm support vector machine. We argue that the 1-norm support vector machine may have some advantage over the standard 2-norm support vector machine, especially when there are redundant noise features. We also propose an efficient algorithm that computes the whole solution path of the 1-norm support vector machine, hence facilitates adaptive selection of the tuning parameter for the 1-norm support vector machine.

Acknowledgments

First and foremost, I wish to express my great appreciation to my advisor Trevor Hastie. He suggested the topic for this thesis and provided invaluable advice and constant encouragement throughout the course of my research. Without his support and guidance, this work would not have been completed. Trevor has been more than an academic advisor to me. When my son was one month old, my wife became infected with mastitis, which resulted in a very difficult time for our family. I still vividly remember how Trevor immediately called us after he heard the news and offered his help. I feel very fortunate to have Trevor as a mentor and a friend. I shall always appreciate his guidance which led me to the wonderful world of statistics.

I am very lucky to have met Saharon Rosset here at Stanford – I now have a talented collaborator. He is a constant inspiration to me and I value our relationship greatly.

I am also very grateful to Professor Rob Tibshirani, whose stimulating suggestions, sharp comments and warm encouragement accompanied me throughout the dissertation process. I owe my gratitude to Professor Susan Holmes and Professor Persi Diaconis – I would probably not have chosen to study statistics or come to Stanford had I not met Susan and Persi when I was at Cornell. I want to thank Susan again for writing me two important recommendation letters at two different stages, one when I applied to the graduate school and the other when I applied for a job. I wish to thank Professor Brad Efron, Professor Jerry Friedman and Professor Art Owen for acting as members of my oral committee and providing useful suggestions on my research.

I also wish to thank my friends and classmates at Stanford, too many to name, and the

Monday fish-bowl group for helping me out in many ways and making my life here colorful and enjoyable. My special thanks go to Xizhao, whose companionship and unselfish support ever since we began our journey together have brightened my life. Finally, I would like to thank my parents for their love and confidence in me. My parents' support is a big part of everything that I accomplish.

Contents

List of Tables

List of Figures

Chapter 1

Introduction

1.1 The Classification Problem

In standard classification problems, we are given a set of training data (x_1, y_1), (x_2, y_2), $\ldots (x_n, y_n)$, where the input (predictor variable) $x_i \in \mathcal{R}^p$ and the output (response variable) y_i is qualitative and assumes values in a finite set, e.g. $\mathcal{C} = \{1, 2, \ldots K\}$. The aim is to find a classification rule from the training data, so that when given a new input x, we can assign a class $c(x)$ from $\mathcal{C}$ to it.

The question, then, is what is the *best* possible classification rule. To answer this question, we need to define what we mean by *best*. A common definition of *best* is to achieve the lowest misclassification error rate. Usually it is assumed that the training data are an independently and identically distributed samples from an unknown probability distribution $P(X, Y)$. Then the misclassification error rate is:

$$
\begin{aligned}
E_{X,Y} 1_{c(X) \neq Y} &= E_X P(c(X) \neq Y | X) && (1.1)\\
&= 1 - E_X P(c(X) = Y | X) && (1.2)\\
&= 1 - \sum_{k=1}^{K} E_X \left[1_{c(X)=k} P(Y = k | X) \right] && (1.3)
\end{aligned}
$$

It is clear that

$$c(x) = \text{argmax}_k P(Y = k|X = x)$$

will minimize this quantity with the misclassification error rate equal to $1 - E_X \max_k P(Y = k|X)$. This classifier is known as the *Bayes classifier*, and the error rate it achieves is the *Bayes error rate*.

Below we see two real classification examples that are discussed in greater detail in the thesis.

1.1.1 Handwritten Digit Recognition

Consider the digits in Figure (1.1) (Reprinted from Hastie, Tibshirani & Friedman (2001)). These are scanned in images of handwritten zip codes from the US Postal Service Zip Code Data Set. The images are 16×16 greyscale maps. We thus have a $p = 256$ dimensional predictor space, $x \in \mathcal{R}^{256}$, and a response space containing 10 possible outcomes, $\mathcal{C} = \{0, 1, \ldots 9\}$. The task is to use a set of training data (pairs of x_i and y_i) to form a rule to predict y (the digit) given x (the pixels).

1.1.2 DNA Microarray Classification

DNA microarrays measure the expression of a gene in a cell by measuring the amount of mRNA present for that gene. Microarrays are considered a breakthrough technology in biology, for they facilitate the quantitative study of thousands of genes simultaneously from a single sample of cells.

Figure (1.2) displays the heat map of a data set containing $2,308$ genes (rows) and 63 samples (columns). The samples are a collection of small round blue cell tumors (SRBCT) from children and each sample belongs to one of four tumor classes. The goal is to predict the diagnostic tumor category of a sample on the basis of its gene expression profile. Here we have a situation that the number of inputs (genes) ($p = 2,308$) is much larger than the number of samples ($n = 63$). Hence, besides predicting the correct tumor class for a

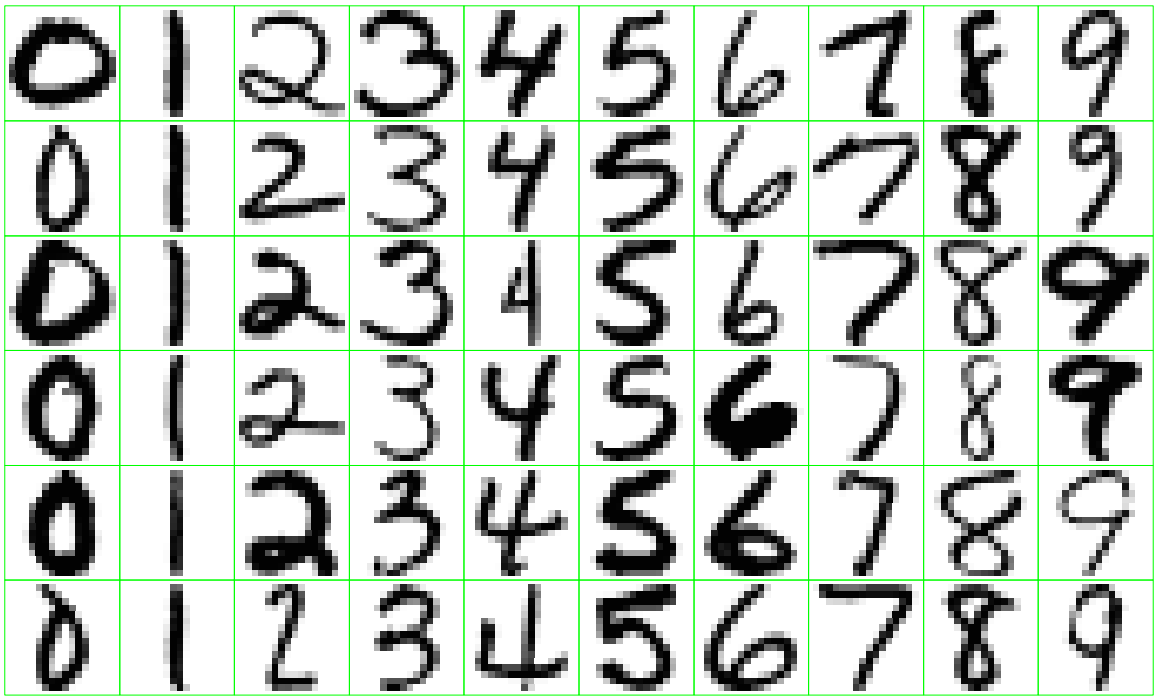

Figure 1.1: *Examples of hand written digits from U.S. postal envelopes.*

given sample, another challenge in microarray diagnosis is to identify relevant genes that contribute most to the classification.

1.2 Support Vector Machines

The support vector machine (SVM) has been a popular tool for classification problems in the machine learning field. Recently, it has also gained increasing attention from the statistics community. Below we briefly go over the support vector machine for two-class classification from two perspectives. See Vapnik (1995), Burges (1998), Evgeniou, Pontil & Poggio (1999) and Hastie, Tibshirani & Friedman (2001) for details.

1.2.1 Margin Maximizer

Recall the training data are (x_1, y_1), (x_2, y_2), $\ldots$ (x_n, y_n), $x_i \in \mathcal{R}^p$. In two-class classification, $y_i \in \{-1, 1\}$. Let us first consider the case when the training data can be perfectly separated

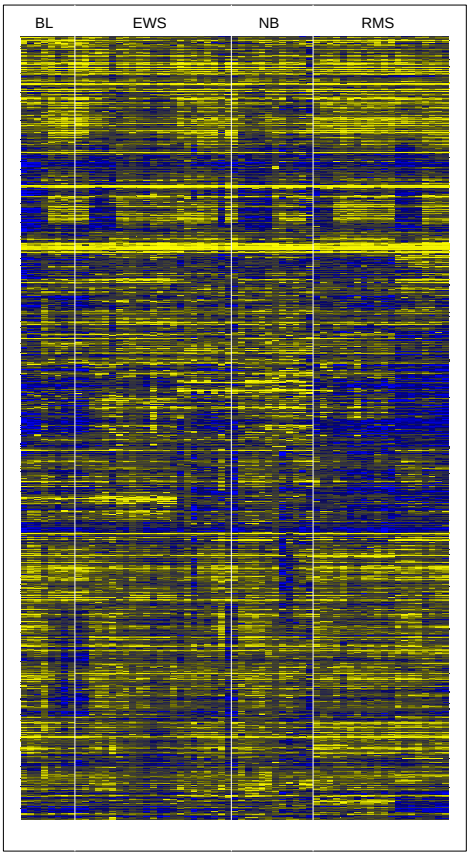

Figure 1.2: *DNA microarray data: expression matrix of 2308 genes (rows) and 64 samples (columns) for the SRBCT data.*

by a hyperplane in $\mathcal{R}^p$. Define the hyperplane by

$$\{x : f(x) = \beta_0 + x^T\beta = 0\},$$

where β is a unit vector: $\|\beta\|_2 = 1$, then $f(x)$ gives the signed distance from a point x to the hyperplane. Since the training data are linearly separable, we are able to find a hyperplane such that

$$y_i f(x_i) > 0 \quad \forall i. \tag{1.4}$$

Indeed, there are infinitely many such hyperplanes. Among the hyperplanes satisfying (1.4), the support vector machine looks for the one that maximizes the *margin*. The margin is defined as the shortest distance from the training data to the hyperplane. Hence we can write the support vector machine problem as:

$$\max_{\beta,\beta_0,\|\beta\|_2=1} \quad C \tag{1.5}$$

$$\text{subject to} \quad y_i(\beta_0 + x_i^T\beta) \geq C, \quad i = 1, \ldots, n \tag{1.6}$$

When the training data are not linearly separable, we allow some training data to be on the wrong side of the edges of margin and introduce slack variables $\xi_i, \xi_i \geq 0$. The support vector machine problem then becomes

$$\max_{\beta,\beta_0,\|\beta\|_2=1} \quad C \tag{1.7}$$

$$\text{subject to} \quad y_i(\beta_0 + x_i^T\beta) \geq C(1 - \xi_i), \quad i = 1, \ldots, n \tag{1.8}$$

$$\xi_i \geq 0, \sum \xi_i \leq B, \tag{1.9}$$

where B is a prespecified positive number, which can be regarded as a *tuning parameter*.

(1.7) – (1.9) can be turned into a quadratic programming problem and the solution has

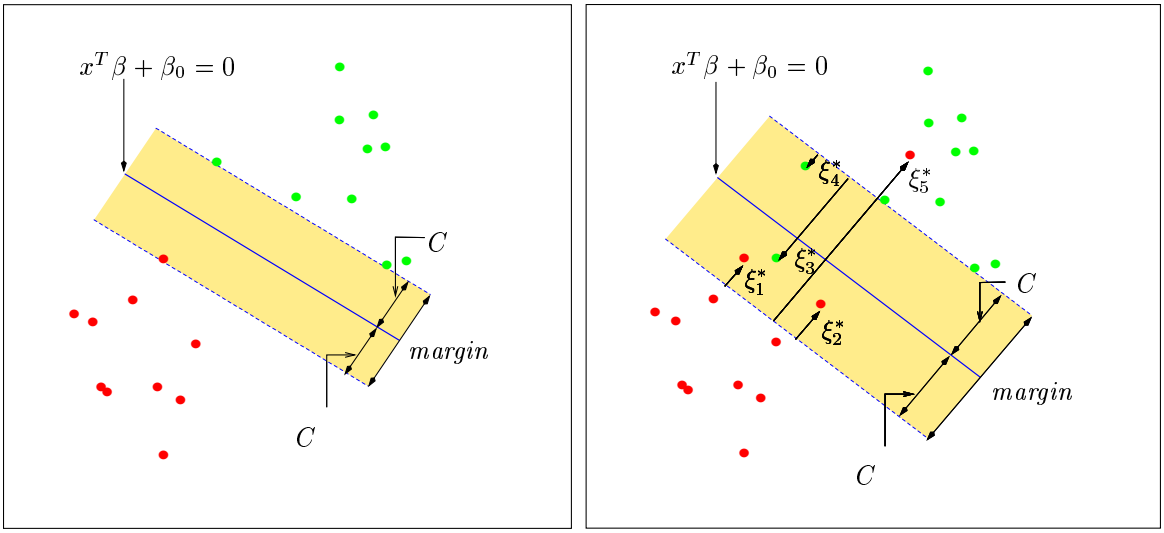

Figure 1.3: *Linear support vector machine classifiers.*

the form:

$$f(x) = \beta_0 + \sum_{i=1}^{n} \alpha_i y_i \langle x_i, x \rangle, \tag{1.10}$$

where α_i's are Lagrangian multipliers for the quadratic programming problem, and

$$\beta = \sum_{i=1}^{n} \alpha_i y_i x_i.$$

Notice each training data has a corresponding α_i. Using the Karush-Kuhn-Tucker (KKT) conditions of the quadratic programming problem, one can show a sizeable fraction of the n values of α_i are zero. This seems to be an attractive property, because only the data points near the classification boundary (including those on the wrong side of the boundary) have an influence in determining the position of the boundary, and hence have non-zero α_i's. The corresponding x_i's are called *support points* (support vectors). Figure (1.3) illustrates both the linearly separable and non-separable cases.

As with other linear models, we can make the support vector machine more flexible by enlarging the feature space using basis expansions such as polynomials or splines. Generally,

linear boundaries in the enlarged space achieve better training data separation and translate to nonlinear boundaries in the original input space. Suppose the dictionary of the basis functions of the enlarged feature space is

$$\mathcal{D} = \{h_1(x), h_2(x), \ldots h_q(x)\},$$

where q is the dimension of the enlarged feature space. Note if $q = p$ and $h_j(x)$ is the jth component of x, the enlarged feature space is reduced to the original input space. The classification boundary in the enlarged feature space is given by

$$\{x : f(x) = \beta_0 + h(x)^T\beta = 0\}.$$

Note that in the linear support vector machine, the solution (1.10) depends on the basis functions only through their inner product. Hence, when using the enlarged basis functions, the new solution will have the form:

$$f(x) = \beta_0 + \sum_{i=1}^{n} \alpha_i y_i \langle h(x_i), h(x)\rangle. \tag{1.11}$$

This implies that if the enlarged basis functions are basis functions of a reproducing kernel Hilbert space (RKHS) (Wahba (1990)), with

$$K(x, x') = \langle h(x), h(x')\rangle,$$

then we do not need to know the enlarged basis functions $h(x)$ explicitly; all we need to know is the kernel function $K(x, x')$ that generates the reproducing kernel Hilbert space. This kernel trick allows the enlarged feature space to be even infinite dimensional, i.e., $q = \infty$, without causing any further computational burden, since the solution (1.11) has a finite dimensional form in terms of the kernel bases $K(x_i, x)$, and the number of parameters is always $n + 1$ (n for α_i's and 1 for β_0). Three popular choices of $K(\cdot, \cdot)$ in the support

vector machine literature are:

$$d\text{th Degree polynomial} \;:\; K(x,x') = \left(1+\langle x,x'\rangle\right)^d, \tag{1.12}$$

$$\text{Radial basis} \;:\; K(x,x') = \exp\left(-\|x-x'\|^2/2\sigma^2\right), \tag{1.13}$$

$$\text{Neural network} \;:\; K(x,x') = \tanh\left(\kappa_1\langle x,x'\rangle + \kappa_2\right), \tag{1.14}$$

where d, σ, κ_1 and κ_2 are pre-specified parameters.

1.2.2 Regularized Function Fitting

Section (1.2.1) views the support vector machine from a geometric point of view, i.e., a hyperplane in an enlarged reproducing kernel Hilbert space that maximizes the margin of the training data. It turns out that the support vector machine is also equivalent to a regularized function fitting problem. With $f(x) = \beta_0 + h(x)^T\beta$, consider the optimization problem:

$$\min_{\beta_0,\beta} \sum_{i=1}^{n} [1-y_i f(x_i)]_+ + \lambda\|\beta\|_2^2 \tag{1.15}$$

where the subscript "+" indicates a positive part and λ is a tuning parameter. One can show that the solution to (1.15) is the same as the support vector machine (1.7) – (1.9). Furthermore, if $h(x)$ are appropriate basis functions of a reproducing kernel Hilbert space, (1.15) can be written as:

$$\min_{f\in\mathcal{H}_K} \sum_{i=1}^{n} [1-y_i f(x_i)]_+ + \lambda\mathcal{J}(f), \tag{1.16}$$

where $\mathcal{H}_K$ is the reproducing kernel Hilbert space, and $\mathcal{J}(f) = \|f\|_{\mathcal{H}_K}^2$ is the square of the L_2 norm of $f(x)$ defined on $\mathcal{H}_K$. Similar to Section (1.2.1), although $\mathcal{H}_K$ can be infinite

dimensional, the solution of (1.16) has a finite dimensional form:

$$f(x) = \beta_0 + \sum_{i=1}^{n} \alpha_i y_i K(x_i, x), \tag{1.17}$$

where $K(\cdot, \cdot)$ is the positive definite kernel function that generates the reproducing kernel Hilbert space $\mathcal{H}_K$.

Notice both (1.15) and (1.16) have the form $loss + penalty$, which is a familiar paradigm to statisticians in function estimation. The loss function $(1 - yf)_+$ is called the *hinge* loss. Lin (2002) shows:

$$\text{argmin}_f E\left[(1 - Yf(x))_+\right] = sign(p_1(x) - 1/2) \quad \text{or} \quad sign\left(\log \frac{p_1(x)}{1 - p_1(x)}\right),$$

where $p_1(x) = P(Y = 1|X = x)$ is the conditional probability of a point being in class 1 given $X = x$. Hence the support vector machine tries to implement the optimal Bayes classification rule without estimating the actual conditional probability $p_1(x)$.

1.2.3 Multi-class Support Vector Machine

The support vector machine classifier so far described is for two-class classification. In going from two-class to multi-class classification, many researchers have proposed various procedures.

In practice, the one-vs-rest scheme is often used: given K classes, the problem is divided into a series of K one-vs-rest problems, and each one-vs-rest problem is addressed by a different class-specific support vector machine classifier (e.g., "class 1" vs. "not class 1"); then a new sample takes the class of the classifier with the largest real valued output $c = \text{argmax}_{k=1,\ldots K} f_k$, where f_k is the real valued output of the kth support vector machine classifier.

Instead of solving K problems, Weston & Watkins (1999) and Vapnik (1998) generalize

(1.7) – (1.9) by solving one single optimization problem:

$$\max_{\beta_k,\beta_{0k}} \quad C \tag{1.18}$$

$$\text{subject to} \quad (\beta_{0y_i} - \beta_{0k}) + x_i^T(\beta_{y_i} - \beta_k) \geq C(1 - \xi_{ik}), \tag{1.19}$$

$$i = 1, \ldots n, \quad k = 1, \ldots K, k \neq y_i \tag{1.20}$$

$$\xi_{ik} \geq 0, \sum_i \sum_{k \neq y_i} \xi_{ik} \leq B \tag{1.21}$$

$$\sum_{k=1}^{K} \|\beta_k\|_2^2 = 1. \tag{1.22}$$

Similar to section 1.2.1, this setup also has a nice geometric interpretation.

Recently, Lee, Lin & Wahba (2002) proposed an algorithm that implements the Bayes classification rule and estimates $\text{argmax}_k P(Y = k|X = x)$ directly, but the geometric picture of their algorithm is still not clear.

1.3 Outline of the Thesis

In Chapter 2 we propose a new approach for classification, called the import vector machine (IVM), which is built on kernel logistic regression (KLR). We show that the import vector machine not only performs as well as the support vector machine in two-class classification, but also can naturally be generalized to the multi-class case. Furthermore, the import vector machine provides an estimate of the underlying probability. Similar to the *support points* of the support vector machine, the import vector machine model uses only a fraction of the training data to index kernel basis functions, typically a much smaller fraction than the support vector machine. This gives the import vector machine a computational advantage over the support vector machine, especially when the size of the training data set is large.

The import vector machine is based on kernel logistic regression, which replaces the hinge loss of the support vector machine with negative binomial log-likelihood. In Chapter 3, we replace the penalty term $\|\beta\|_2^2$, the square of the L_2 norm of β, in (1.15) of the support vector machine, with the L_1 norm $\|\beta\|_1$, and we fit the 1-norm support vector machine. The

motivation for doing such a replacement is that in addition to shrinking the fitted coefficients $\hat{\beta}$ towards zero, just as the L_2 penalty does, the L_1 penalty also tends to set some of the fitted coefficients exactly equal to zero. Hence the 1-norm support vector machine fits a model that does automatic feature selection. We show that the fitted coefficients path $\hat{\beta}$ as a function of the tuning parameter λ is piece-wise linear, and we give an efficient algorithm that computes the whole coefficients path. This facilitates efficient adaptive selection of the tuning parameter λ.

In Chapter 4, we concentrate on DNA microarray classification using techniques developed in Chapter 2 and Chapter 3. Often a primary goal in microarray diagnosis is to identify the genes responsible for the classification, rather than class prediction. Besides the automatic gene selection done by the L_1 penalty as described in Chapter 3, we consider two gene selection methods used in the literature, univariate ranking (UR) and recursive feature elimination (RFE). Empirical results indicate that recursive feature elimination method tends to select fewer genes than other methods and also performs well in both cross-validation and test data.

In the Appendix we give proofs for all the theorems contained in the thesis.

Chapter 2

Import Vector Machines

In this chapter, we propose a new approach for classification, the import vector machine (IVM), that is built on kernel logistic regression (KLR). We show that the import vector machine not only performs as well as the support vector machine in two-class classification, but also can naturally be generalized to the multi-class case. Furthermore, the import vector machine provides an estimate of the underlying probability. Similar to the *support points* of the support vector machine, the import vector machine model uses only a fraction of the training data to index kernel basis functions, typically a much smaller fraction than the support vector machine. This gives the import vector machine a potential computational advantage over the support vector machine, especially when the size of the training data set is large.

2.1 Kernel Logistic Regression

As described in Chapter 1, the standard support vector machine produces a non-linear classification boundary in the original input space by constructing a linear boundary in an enlarged version of the original input space. The dimension of the enlarged space can be very large, even infinite, in some cases. This seemingly prohibitive computation is achieved through a positive definite reproducing kernel $K(\cdot, \cdot)$, which gives the inner product in the enlarged space.

In Chapter 1 we have also noted the relationship between the support vector machine and regularized function fitting in the reproducing kernel Hilbert spaces (RKHS). An overview can be found in Evgeniou, Pontil & Poggio (1999), Wahba (1999) and Hastie, Tibshirani & Friedman (2001). Fitting a support vector machine is equivalent to:

$$\min_{f \in \mathcal{H}_K} \sum_{i=1}^{n} [1 - y_i f(x_i)]_+ + \lambda \|f\|^2_{\mathcal{H}_K}, \tag{2.1}$$

where $\mathcal{H}_K$ is the reproducing kernel Hilbert space generated by a kernel $K(\cdot, \cdot)$. By the representer theorem (Kimeldorf & Wahba (1971)), the optimal $f(x)$ has the form:

$$f(x) = \beta_0 + \sum_{i=1}^{n} \alpha_i K(x_i, x), \tag{2.2}$$

and only the support points will have non-zero α_i's.

Note that (2.1) has the form $loss + penalty$. The loss function $(1 - yf)_+$ is plotted in Figure 2.1, along with several traditional loss functions. As we can see, the negative log-likelihood (NLL) of the binomial distribution has a shape similar to that of the support vector machine: both increase linearly as yf gets very small (negative) and encourage y and f to have the same sign. If we replace $(1-yf)_+$ in (2.1) with $\ln(1+e^{-yf})$, the negative log-likelihood of the binomial distribution, the problem becomes a kernel logistic regression problem:

$$\min_{f \in \mathcal{H}_K} \sum_{i=1}^{n} \ln\left(1 + e^{-y_i f(x_i)}\right) + \lambda \|f\|^2_{\mathcal{H}_K}. \tag{2.3}$$

Because of the similarity between the two loss functions, we expect that the fitted function performs similarly to the support vector machine for two-class classfication.

There are two immediate advantages of making such a replacement: (a) Kernel logistic

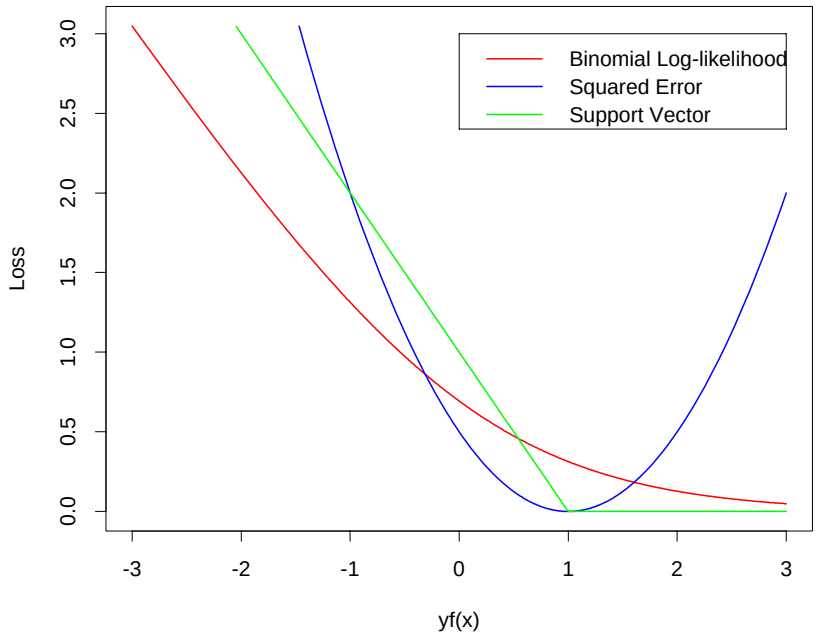

Figure 2.1: *Some loss functions.* $y \in \{-1, 1\}$.

regression estimates the log-odds:

$$f(x) = \log \frac{P(Y=1|X=x)}{P(Y=-1|X=x)} \tag{2.4}$$

$$= \beta_0 + \sum_{i=1}^{n} \alpha_i K(x_i, x). \tag{2.5}$$

Hence, besides giving a classification rule, kernel logistic regression also offers a natural estimate of the probability

$$p_1(x) = \frac{e^{f(x)}}{1 + e^{f(x)}},$$

while the support vector machine only estimates (Lin (2002))

$$sign\left[p_1(x) - 1/2\right] \quad \text{or} \quad sign\left[\log \frac{p_1(x)}{1 - p_1(x)}\right],$$

where $p_1(x) \equiv P(Y = 1|X = x)$ is the conditional probability of a point being in class 1

given $X = x$; (b) The kernel logistic regression can naturally be generalized to the multi-class case through kernel multi-logit regression, whereas this is not the case for the support vector machine. However, because the kernel logistic regression compromises the hinge loss function of the support vector machine, it no longer has the *support points* property; in other words, all the α_i's in (2.5) are non-zero.

Kernel logistic regression is a well-studied problem; see Wahba, Gu, Wang & Chappell (1995), Green & Yandell (1985), Hastie & Tibshirani (1990) and references therein; however, they are all under the smoothing spline analysis of variance scheme.

We use a simulation example to illustrate the similar performance of kernel logistic regression and the support vector machine. The data in each class are simulated from a mixture of Gaussian distribution (Hastie, Tibshirani & Friedman (2001)): first we generate 10 means μ_k from a bivariate Gaussian distribution $N((1,0)^T, \mathbf{I})$ and label this class $+1$. Similarly, 10 more are drawn from $N((0,1)^T, \mathbf{I})$ and labeled class -1. Then for each class, we generate 100 observations as follows: for each observation, we pick an μ_k at random with probability $1/10$, and then generate a $N(\mu_k, \mathbf{I}/5)$, thus leading to a mixture of Gaussian clusters for each class.

We use a radial basis kernel (1.13). The regularization parameter λ is chosen to achieve good misclassification error. The results are shown in Figure 2.2. The radial basis kernel produces a boundary quite close to the Bayes optimal boundary for this simulation. We see that the fitted model of kernel logistic regression is quite similar in classification performance to that of the support vector machine. In addition to a classification boundary, since kernel logisitic regression estimates the log-odds of class probabilities, it can also produce probability contours (Figure 2.2).

2.1.1 KLR as Margin Maximizer

Recall that in Chapter 1 we described the support vector machine from two perspectives: (a) geometrically as the margin maximizer, and (b) as a regularized function fitting problem. The motivation for fitting a kernel logistic regression model is the similarity in shape between the negative log-likelihood of the binomial distribution and the hinge loss of the support

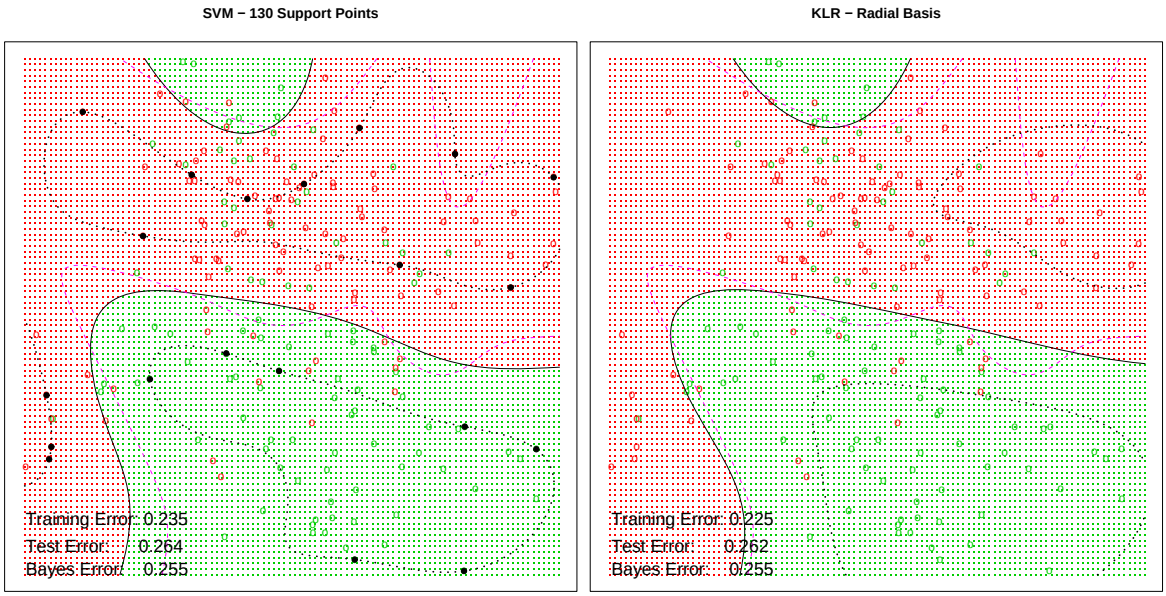

Figure 2.2: *The solid black lines are classification boundaries; the dashed purple lines are Bayes optimal boundaries. For the SVM, the dashed black lines are the edges of the margins and the black points are the points exactly on the edges of the margin. For KLR, the dashed black lines are the* $p_1(x) = 0.25$ *and* 0.75 *lines.*

vector machine, and this motivation is from the regularized function fitting perspective. Since the support vector machine was initiated as a method to maximize the margin of the training data, then a natural question is: what does kernel logistic regression do with the margin? It turns out that kernel logistic regression can also be regarded as a margin maximizer.

Similar to section 1.2.1, let

$$\mathcal{D} = \{h_1(x), h_2(x), \dots h_q(x)\}$$

be the dictionary of the basis functions of the enlarged feature space, where q is the dimension of the enlarged feature space. The classification boundary, a hyperplane in this enlarged feature space, is given by:

$$\{x : f(x) = \beta_0 + h(x)^T\beta = 0\}.$$

Suppose the enlarged feature space is so rich that the training data are separable, then the margin-maximing support vector machine can be written as:

$$\max_{\beta_0,\beta,\|\beta\|_2=1} \quad C \tag{2.6}$$

$$\text{subject to} \quad y_i\left(\beta_0 + h(x_i)^T\beta\right) \geq C, \quad i = 1, \ldots, n \tag{2.7}$$

where C is the shortest distance from the training data to the separating hyperplane and defined as the *margin*.

Now consider an equivalent setup of kernel logistic regression:

$$\min_{\beta_0,\beta} \quad \sum_{i=1}^{n} \ln\left(1 + e^{-y_i f(x_i)}\right) \tag{2.8}$$

$$\text{subject to} \quad \|\beta\|_2^2 \leq s \tag{2.9}$$

$$f(x_i) = \beta_0 + h(x_i)^T\beta, \quad i = 1, \ldots, n. \tag{2.10}$$

Then we have the following theorem:

Theorem 2.1 *Suppose the training data are separable, i.e.* $\exists\beta_0, \beta$, *s.t.* $y_i(\beta_0 + h(x_i)^T\beta) > 0$, $\forall i$. *Let the solution of (2.8)-(2.10) be denoted by* $\hat{\beta}(s)$, *then*

$$\frac{\hat{\beta}(s)}{s} \to \beta^* \quad as \quad s \to \infty,$$

where β^* *is the solution of the margin-maximizing support vector machine (2.6)-(2.7), if* β^* *is unique.*

If β^* *is not unique, then* $\frac{\hat{\beta}(s)}{s}$ *may have multiple convergence points, but they will all represent margin-maximizing separating hyperplanes.*

The proof of the theorem is delayed in the Appendix. Theorem 2.1 implies that kernel logistic regression, similar to the support vector machine, is also a kind of margin maximizer.

2.1.2 Newton-Raphson Method

In this section, we describe how to solve kernel logistic regression using the Newton-Raphson method. Let

$$H = \sum_{i=1}^{n} \ln\left(1 + e^{-y_i f(x_i)}\right) + \frac{\lambda}{2}\|f\|^2_{\mathcal{H}_K}. \tag{2.11}$$

Then, by the representer theorem (Kimeldorf & Wahba (1971)), the solution that minimizes H has the form:

$$f(x) = \beta_0 + \sum_{i=1}^{n} \alpha_i K(x_i, x).$$

Let

$$\begin{aligned}
p_i &= \frac{1}{1 + e^{y_i f(x_i)}}, \quad i = 1, \ldots n && (2.12)\\
\vec{\alpha} &= (\beta_0, \alpha_1, \ldots \alpha_n)^T && (2.13)\\
\vec{p} &= (p_1, \ldots p_n)^T && (2.14)\\
\vec{y} &= (y_1, \ldots, y_n)^T && (2.15)\\
K_1 &= \left(\vec{1}, K(x_i, x_{i'})_{i,i'=1}^{n}\right) && (2.16)\\
K_2 &= \begin{pmatrix} 0 & 0 \\ 0 & K(x_i, x_{i'})_{i,i'=1}^{n} \end{pmatrix} && (2.17)\\
W &= \operatorname{diag}\left(p_1(1-p_1), \ldots p_n(1-p_n)\right) && (2.18)
\end{aligned}$$

Notice K_1 is a $n \times (n+1)$ matrix, K_2 is a $(n+1) \times (n+1)$ matrix and W is a $n \times n$ matrix. With some abuse of notation, (2.11) can be written in a finite dimensional form:

$$H = \vec{1}^T \ln\left(1 + e^{-\vec{y}\cdot(K_1\vec{\alpha})}\right) + \frac{\lambda}{2}\vec{\alpha}^T K_2 \vec{\alpha}. \tag{2.19}$$

Now we have

$$\frac{\partial H}{\partial \vec{\alpha}} = -K_1^T(\vec{y} \cdot \vec{p}) + \lambda K_2 \vec{\alpha} \tag{2.20}$$

$$\frac{\partial^2 H}{\partial \vec{\alpha} \partial \vec{\alpha}^T} = K_1^T W K_1 + \lambda K_2 \tag{2.21}$$

where "$\cdot$" denotes element-wise multiplication.

The Newton-Raphson method then iterates as Algorithm 2.1:

Algorithm 2.1 *Newton-Raphson Method for KLR*

1. Initialize $\vec{\alpha}_0$.

2. Compute $\vec{p}, K_1, K_2$ and W.

3. Update

$$\vec{\alpha}_t = \vec{\alpha}_{t-1} - \left(\frac{\partial^2 H}{\partial \vec{\alpha} \partial \vec{\alpha}^T}\right)^{-1} \frac{\partial H}{\partial \vec{\alpha}} \tag{2.22}$$

$$= \vec{\alpha}_{t-1} + \left(K_1^T W K_1 + \lambda K_2\right)^{-1} \left(K_1^T(\vec{y} \cdot \vec{p}) - \lambda K_2 \vec{\alpha}_{t-1}\right) \tag{2.23}$$

$$= \left(K_1^T W K_1 + \lambda K_2\right)^{-1} K_1^T W \vec{z} \tag{2.24}$$

where

$$\vec{z} = K_1 \vec{\alpha}_{t-1} + W^{-1}(\vec{y} \cdot \vec{p})$$

4. Repeat steps (2) and (3) until $\vec{\alpha}_t$ converges.

In (2.24), we have re-expressed the Newton-Raphson step as a weighted least-squares step. $\vec{z}$ is sometimes known as the adjusted response, and the step is referred to as *iteratively reweighted least squares.* $\vec{\alpha}_0 = 0$ is usually a good starting value. Since H is convex, the algorithm typically does converge, but overshooting can occur. In the rare case that overshooting occurs, step size halving will gurantee convergence.

2.1.3 Sequential Minimal Optimization Method

The drawback of the Newton-Raphson method is that in each iteration, an $n \times n$ matrix needs to be inverted. The corresponding computational cost can be high when n is large.

Recently, Keerthi, Duan, Shevade & Poo (2002) proposed a dual algorithm for kernel logistic regression that avoids inverting huge matrices. It follows the spirit of the popular sequential minimal optimization (SMO) algorithm (Platt (1999)).

Let $C = 1/\lambda$. Let

$$F_i = \sum_{i'=1}^{n} \alpha_{i'} y_{i'} K(x_{i'}, x_i) + y_i \ln\left(\frac{\alpha_i}{C - \alpha_i}\right) \tag{2.25}$$

$$\begin{aligned} f(\alpha) &= \frac{1}{2} \sum_i \sum_{i'} \alpha_i \alpha_{i'} y_i y_{i'} K(x_i, x_{i'}) \\ &+ C \sum_i \frac{\alpha_i}{C} \ln \frac{\alpha_i}{C} + (1 - \frac{\alpha_i}{C}) \ln\left(1 - \frac{\alpha_i}{C}\right) \end{aligned} \tag{2.26}$$

Then, the sequential minimal optimization algorithm proceeds as Algorithm 2.2:

Algorithm 2.2 *Sequential Minimal Optimization Algorithm for KLR*

1. Initialize α_i such that

$$0 < \alpha_i < C \quad \text{and} \quad \sum_{i=1}^{n} \alpha_i y_i = 0.$$

2. Let

$$F_{up} = \max_i F_i \tag{2.27}$$

$$F_{low} = \min_i F_i \tag{2.28}$$

$$i_{up} = \text{argmax}_i F_i \tag{2.29}$$

$$i_{low} = \text{argmin}_i F_i \tag{2.30}$$

If $F_{up} = F_{low}$, stop. If not, let

$$\tilde{\alpha}_{i_{up}}(s) = \alpha_{i_{up}} - s/y_{i_{up}} \tag{2.31}$$

$$\tilde{\alpha}_{i_{low}}(s) = \alpha_{i_{low}} + s/y_{i_{low}} \tag{2.32}$$

$$\tilde{\alpha}_i(s) = \alpha_i \quad \forall i \neq i_{up}, i_{low} \tag{2.33}$$

Find s^* that minimize $f\left(\tilde{\alpha}(s)\right)$.

3. Update

$$\alpha \leftarrow \tilde{\alpha}(s^*).$$

Go to step (2).

At the end of the algorithm, we have $\beta_0 = -F_{up} = -F_{low}$ and

$$f(x) = \beta_0 + \sum_{i=1}^{n} \alpha_i y_i K(x_i, x) \tag{2.34}$$

Preliminary computational experiments show that this sequential minimal optimization algorithm is robust and fast (Keerthi, Duan, Shevade & Poo (2002)). We have generalized this algorithm to the multi-class case. A detailed description of the multi-class case algorithm is given in the Appendix.

2.2 Import Vector Machines

Although the sequential minimal optimization method helps reduce the computational cost of kernel logistic regression, in the fitted model

$$f(x) = \beta_0 + \sum_{i=1}^{n} \alpha_i K(x_i, x), \tag{2.35}$$

as mentioned in section 2.1, all the α_i's are non-zero. This is not the case for the support vector machine, for only support points have non-zero α_i's. So the support vector machine allows for data compression and has the advantage of less storage and quicker evaluation.

In this section, we propose an import vector machine model that finds a sub-model to approximate the full model (2.35) given by kernel logistic regression. The sub-model has the form:

$$f(x) = \beta_0 + \sum_{x_i \in \mathcal{S}} \alpha_i K(x_i, x) \tag{2.36}$$

where $\mathcal{S}$ is a subset of the training data $\{x_1, x_2, \ldots x_n\}$, and the data in $\mathcal{S}$ are called *import points*. The advantage of this sub-model is that the computational cost is reduced, especially for large training data sets, while not jeopardizing the performance in classification; and since only a subset of the training data are used to index the fitted model, data compression is achieved.

Several other researchers have also investigated techniques in selecting the subset $\mathcal{S}$. Lin, Wahba, Xiang, Gao, Klein & Klein (2000) divide the training data into several clusters, then randomly select a representative from each cluster to make up $\mathcal{S}$. Smola & Schölkopf (2000) develope a greedy technique to sequentially select m columns of the kernel matrix $[K(x_i, x_{i'})]_{n\times n}$, such that the span of these m columns approximates the span of $[K(x_i, x_{i'})]_{n\times n}$ well in the Frobenius norm. Williams & Seeger (2001) propose randomly selecting m points of the training data, then using the Nystrom method to approximate the eigen-decomposition of the kernel matrix $[K(x_i, x_{i'})]_{n\times n}$, and expanding the results back up to n dimensions. None of these methods uses the output y_i in selecting the subset $\mathcal{S}$ (i.e., the procedure involves only x_i). The import vector machine algorithm uses both the output y_i and the input x_i to select the subset $\mathcal{S}$ in such a way that the resulting fit approximates the full model well.

2.2.1 Algorithm

As mentioned before, we want to find a subset $\mathcal{S}$ of $\{x_1, x_2, \ldots x_n\}$, such that the sub-model (2.36) is a good approximation of the full model (2.35). Since it is computationally impossible to search for every subset $\mathcal{S}$, we use a greedy forward strategy as described in Algorithm 2.3. We call the points in $\mathcal{S}$ *import points*.

Algorithm 2.3 *Basic IVM Algorithm*

1. Let $\mathcal{S} = \emptyset$, $\mathcal{R} = \{x_1, x_2, \ldots, x_n\}$, $t = 1$.

2. For each $x_l \in \mathcal{R}$, let

$$f_l(x) = \beta_0 + \sum_{x_i \in \mathcal{S} \cup \{x_l\}} \alpha_i K(x_i, x)$$

   Find $\vec{\alpha}$ to minimize

$$\begin{aligned} H(x_l) &= \sum_{i=1}^{n} \ln\left(1 + \exp(-y_i f_l(x_i))\right) + \frac{\lambda}{2}\|f_l(x)\|^2_{\mathcal{H}_K} && (2.37) \\ &= \vec{1}^T \ln\left(1 + \exp(-\vec{y} \cdot (K_1^l \vec{\alpha}))\right) + \frac{\lambda}{2}\vec{\alpha}^T K_2^l \vec{\alpha} && (2.38) \end{aligned}$$

   where the regressor matrix

$$K_1^l = [\vec{1}, K(x_i, x_{i'})]_{n \times (m+1)}, \; x_i \in \{x_1, \ldots x_n\}, x_{i'} \in \mathcal{S} \cup \{x_l\};$$

   the regularization matrix

$$K_2^l = \begin{pmatrix} 0 & 0 \\ 0 & K(x_i, x_{i'}) \end{pmatrix}_{(m+1)\times(m+1)}, x_i, x_{i'} \in \mathcal{S} \cup \{x_l\};$$

   and $m = |\mathcal{S}|$.

3. Find

$$x_{l^*} = \text{argmin}_{x_l \in \mathcal{R}} H(x_l).$$

Let $\mathcal{S} = \mathcal{S} \cup \{x_{l^*}\}$, $\mathcal{R} = \mathcal{R} \setminus \{x_{l^*}\}$, $H_t = H(x_{l^*})$, $t \to t+1$.

4. Repeat steps (2) and (3) until H_t converges.

Algorithm 2.3 is computationally feasible, but in step (2) we need to use the Newton-Raphson method or sequential minimal optimization method to find $\vec{\alpha}$ iteratively. When the number of import points m becomes large, the computation can be expensive. To reduce this computation, we use a further approximation.

Instead of iteratively computing $\vec{\alpha}$ until it converges, we can simply do a one-step Newton-Raphson iteration, and use it as an approximation to the converged one. To get a good approximation, we take advantage of the fitted result from the current "optimal" $\mathcal{S}$, i.e., the sub-model when $|\mathcal{S}| = m$, and use it as the initial value. This one-step update is similar to the score test in generalized linear models (GLM), but the latter does not have a penalty term. The updating formula allows the weighted regression (2.24) to be computed in $O(nm)$ time. Hence, we have the revised step (2) of Algorithm 2.3 in Algorithm **??**.

Algorithm 2.4 *Revised Step (2)*

(2^*) For each $x_l \in \mathcal{R}$, correspondingly augment K_1 with a column, and K_2 with a column and a row. Use the updating formula to find $\vec{\alpha}$ in (2.24). Compute (2.37).

In step (4) of Algorithm 2.3, we need to decide when to stop the algorithm. A natural stopping rule is to look at the regularized negative log-likelihood. Let $H_1, H_2, \ldots$ be the sequence of regularized negative log-likelihood obtained in step (3). At each step t, we compare H_t with $H_{t-\Delta t}$, where Δt is a pre-chosen small integer, for example, $\Delta t = 1$. If the ratio $\frac{|H_t - H_{t-\Delta t}|}{|H_t|}$ is less than some pre-chosen small number ϵ, for example, $\epsilon = 0.001$, we stop adding new import points to $\mathcal{S}$.

2.2.2 Selection of λ

So far, we have assumed that the tuning parameter λ is fixed. In practice, we also need to choose an "optimal" λ. We can randomly split all the data into a training set and a tuning set, and use the misclassification error on the tuning set as a criterion for choosing

λ. To reduce the computation, we take advantage of the fact that the regularized negative log-likelihood converges faster for a larger λ. Thus, instead of running the entire revised algorithm for each λ, we propose Algorithm **??**, which combines both adding import points to $\mathcal{S}$ and choosing the optimal λ.

Algorithm 2.5 *Simultaneous Selection of $\mathcal{S}$ and λ*

1. Start with a large tuning parameter λ.
2. Let $\mathcal{S} = \emptyset$, $\mathcal{R} = \{x_1, \ldots, x_n\}$, $t = 1$.
3. Run steps (2^*), (3) and (4) of the revised Algorithm 2.4, until the stopping criterion is satisfied at $\mathcal{S} = \{x_{i_1}, \ldots, x_{i_t}\}$. Along the way, also compute the misclassfication error on the tuning set.
4. Decrease λ to a smaller value.
5. Repeat steps (3) and (4), starting with $\mathcal{S} = \{x_{i_1}, \ldots, x_{i_t}\}$.

We choose the optimal λ as the one that corresponds to the minimum misclassification error on the tuning set.

2.2.3 Simulation Results

In this section, we use a simulation to illustrate the import vector machine method. The data are generated in the same way as Figure 2.2. The simulation results are shown in Figure 2.3 – Figure 2.5.

Figure 2.3 shows how the tuning parameter λ is selected. The optimal λ is found to be equal to 1 and corresponds to a misclassification rate 0.262. Figure 2.4 fixes the tuning paramter to $\lambda = 1$ and finds 19 import points. Figure 2.5 compares the results of the support vector machine and the import vector machine: the support vector machine has 130 support points, and the import vector machine uses 19 import points; they give similar classification boundaries. Figure 2.6 is for the same simulation but different sizes of training data: $n = 200, 400, 600, 800$. We see that as the training data size n increases, the number of import points does not tend to increase.

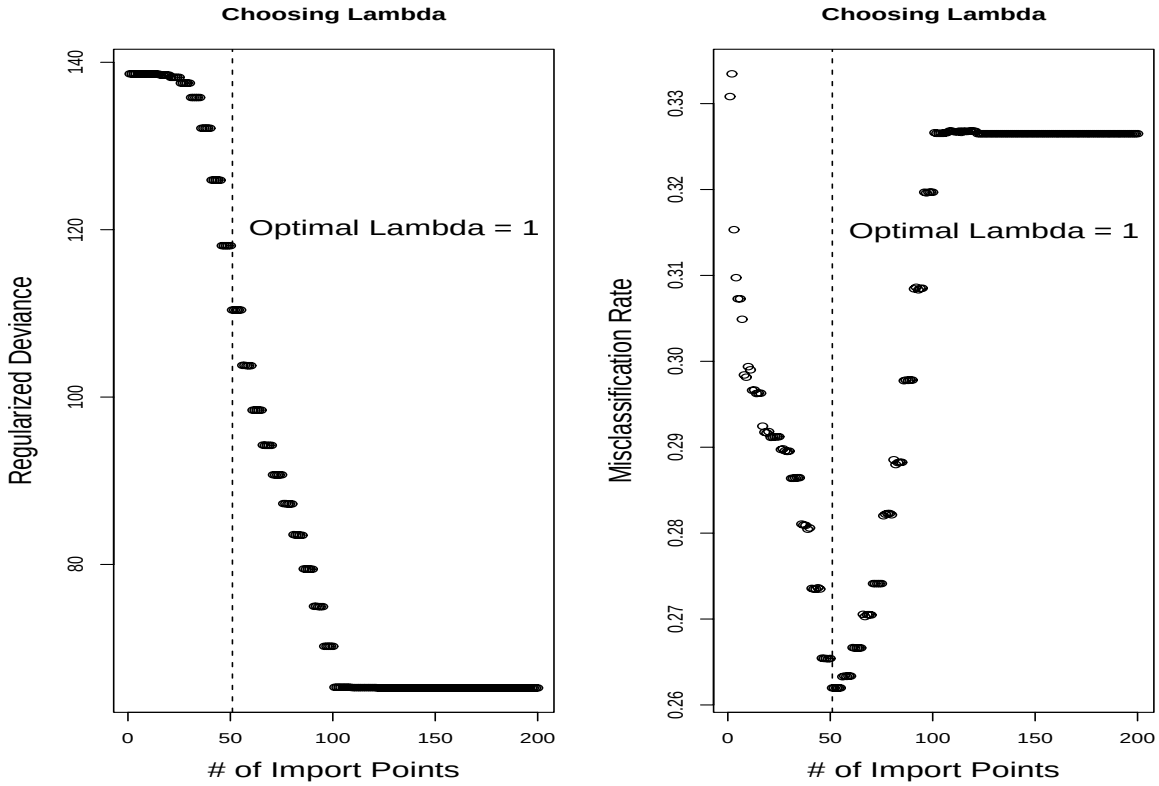

Figure 2.3: *Radial kernel is used.* $n = 200$, $\sigma^2 = 0.7$, $\Delta t = 3$, $\epsilon = 0.001$, λ *decreases from* e^{10} *to* e^{-10}. *The minimum misclassification rate* 0.262 *is found to correspond to* $\lambda = 1$.

Remark: The support points of the support vector machine are those which are close to the classification boundary or misclassified and usually have large weights $[p(x)(1-p(x))]$. The import points of the import vector machine are those that decrease the regularized negative log-likelihood the most, and can be either close to or far from the classification boundary. The difference in properties is natural, because the support vector machine is only concerned with the classification $sign[p(x) - 1/2]$, while the import vector machine also focuses on the unknown probability $p(x)$. Though points away from the classification boundary do not contribute to determining the position of the classification boundary, they may contribute to estimating the unknown probability $p(x)$. The total computational cost of the support vector machine is $O(n^2 s)$, where s is the number of support points, while the computational cost of the import vector machine method is $O(n^2 m^2)$, where m is the number of import points. Since m does not tend to increase as n increases, as illustrated in

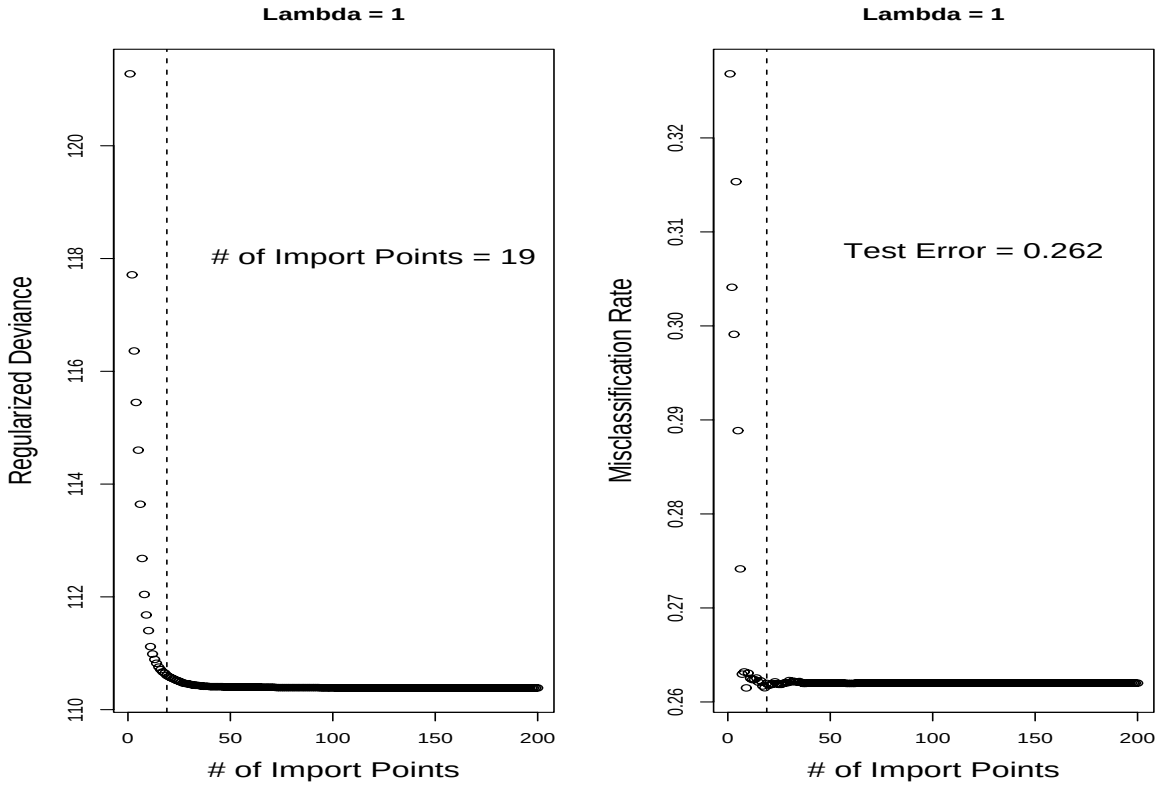

Figure 2.4: *Radial kernel is used.* $n = 200$, $\sigma^2 = 0.7$, $\Delta t = 3$, $\epsilon = 0.001$, $\lambda = 1$. *The stopping criterion is satisfied when* $|\mathcal{S}| = 19$.

Figure 2.6, the computational cost of the import vector machine can be smaller than that of the support vector machine, especially for large training data sets.

2.2.4 Real Data Results

In this section, we compare the performance of the import vector machine and the support vector machine on some real datasets. Ten benchmark datasets are used for this purpose: Banana, Breast-cancer, Flare-solar, German, Heart, Image, Ringnorm, Splice, Thyroid, Titanic, Twonorm and Waveform. Detailed information about these datasets can be found in Rätsch, T.Onoda & K.R.Müller (2000) or are available at $http://ida.first.gmd.edu/\sim raetsch/data$.

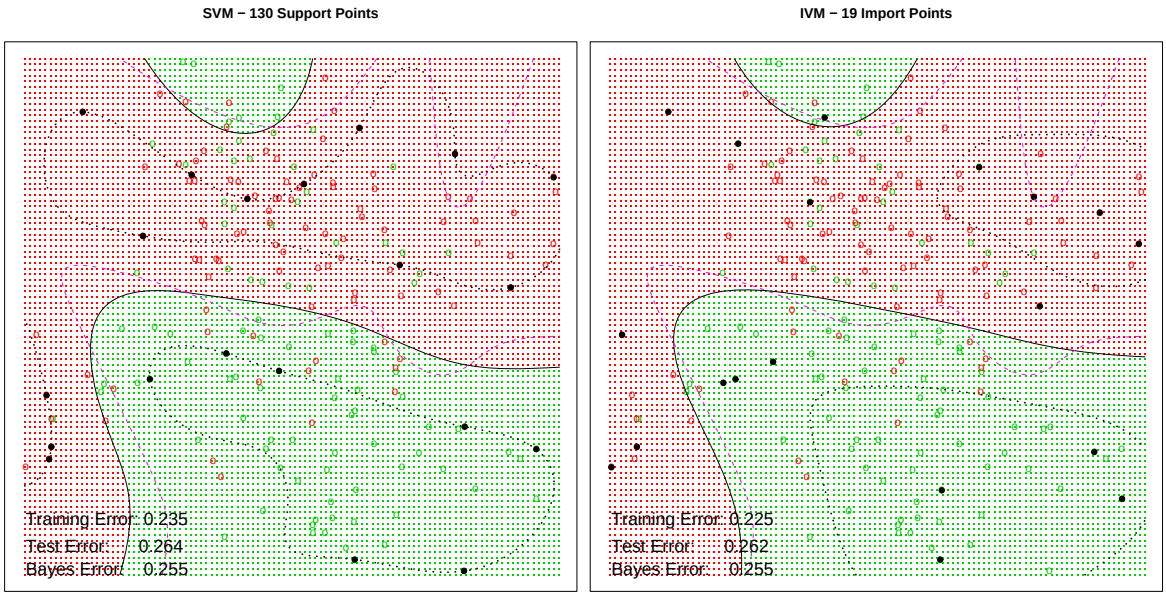

Figure 2.5: *The solid black lines are classification boundaries; the dashed purple lines are Bayes optimal boundaries. For the SVM, the dashed black lines are the edges of the margins, and the black points are the support points exactly on the margin. For the IVM, the dashed black lines are the* $p_1(x) = 0.25$ *and* 0.75 *lines, and the black points are the import points.*

Table 2.1 contains a summary of these datasets. Radial kernel (1.13)

$$K(x, x') = e^{-\frac{\|x-x'\|^2}{2\sigma^2}}$$

is used throughout these datasets. The parameters σ and λ are fixed at specific values that are optimal for the support vector machine's generalization performance (Rätsch, T.Onoda & K.R.Müller (2000)). Each dataset has 20 realizations of the training and test data. The results are in Table 2.2 and Table 2.3. The number outside each bracket is the mean over 20 realizations of the training and test data, and the number in each bracket is the standard deviation. From Table 2.2, we can see that the import vector machine performs as well as the support vector machine in classification on these benchmark datasets. From Table 2.3, we can see that the import vector machine typically uses a much smaller fraction of the training data than the support vector machine to index kernel basis functions. This may give the import vector machine a computational advantage over the support vector

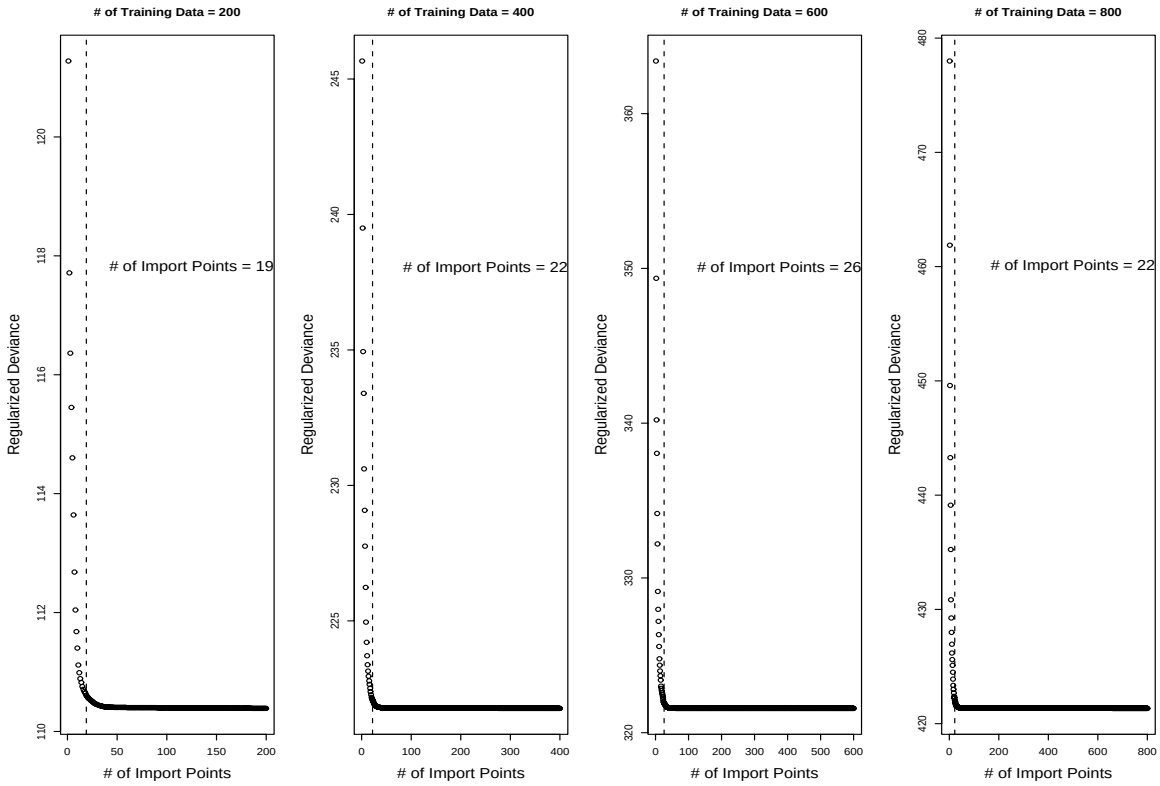

Figure 2.6: *The data are generated in the same way as Figure 2.3 – Figure 2.5. Radial kernel is used.* $\sigma^{=}0.7$, $\lambda = 1$, $\Delta t = 3$, $\epsilon = 0.001$. *The sizes of training data are* $n = 200, 400, 600, 800$, *and the corresponding numbers of import points are* $19, 22, 26, 22$.

machine, especially when the size of the training data is large.

2.3 Multi-class Case

In this section, we briefly describe a generalization of the import vector machine to multi-class classification. Suppose there are K classes. The conditional probability of a point being in class k given $X = x$ is denoted as $p_k(x) = P(Y = k|X = x)$. Hence the Bayes classification rule is given by:

$$c(x) = \text{argmax}_{k\in\{1,\ldots,K\}} p_k(x)$$

Table 2.1: Summary of the ten benchmark datasets. n is the size of the training data, p is the dimension of the original input, σ^2 is the parameter of the radial kernel, λ is the tuning parameter, and N is the size of the test data.

Dataset	n	p	σ^2	λ	N
Banana	400	2	1	3.16×10^{-3}	4900
Breast-cancer	200	9	50	6.58×10^{-2}	77
Flare-solar	666	9	30	0.978	400
German	700	20	55	0.316	300
Heart	170	13	120	0.316	100
Image	1300	18	3	0.002	1010
Ringnorm	400	20	10	10^{-9}	7000
Thyroid	140	5	3	0.1	75
Titanic	150	3	2	10^{-5}	2051
Twonorm	400	20	40	0.316	7000
Waveform	400	21	20	1	4600

Table 2.2: Comparison of classification performance of SVM and IVM on ten benchmark datasets.

Dataset	SVM Error (%)	IVM Error (%)
Banana	10.78($\pm$0.68)	10.34($\pm$0.46)
Breast-cancer	25.58($\pm$4.50)	25.92($\pm$4.79)
Flare-solar	32.65($\pm$1.42)	33.66($\pm$1.64)
German	22.88($\pm$2.28)	23.53($\pm$2.48)
Heart	15.95($\pm$3.14)	15.80($\pm$3.49)
Image	3.34(0.70)	3.31($\pm$0.80)
Ringnorm	2.03($\pm$0.19)	1.97($\pm$0.29)
Thyroid	4.80($\pm$2.98)	5.00($\pm$3.02)
Titanic	22.16($\pm$0.60)	22.39($\pm$1.03)
Twonorm	2.90($\pm$0.25)	2.45($\pm$0.15)
Waveform	9.98($\pm$0.43)	10.13($\pm$0.47)

Table 2.3: Comparison of number of kernel basis used by SVM and IVM on ten benchmark datasets.

Dataset	# of SV	# of IV
Banana	90(±10)	21(±7)
Breast-cancer	115(±5)	14(±3)
Flare-solar	597(±8)	9(±1)
German	407(±10)	17(±2)
Heart	90(±4)	12(±2)
Image	221(±11)	72(±18)
Ringnorm	89(±5)	72(±30)
Thyroid	21(±2)	22(±3)
Titanic	69(±9)	8(±2)
Twonorm	70(±5)	24(±4)
Waveform	151(±9)	26(±3)

The model has the form:

$$p_1(x) = \frac{e^{f_1(x)}}{\sum_{k=1}^{K} e^{f_k(x)}}, \tag{2.39}$$

$$p_2(x) = \frac{e^{f_2(x)}}{\sum_{k=1}^{K} e^{f_k(x)}}, \tag{2.40}$$

$$\vdots \tag{2.41}$$

$$p_K(x) = \frac{e^{f_K(x)}}{\sum_{k=1}^{K} e^{f_k(x)}}, \tag{2.42}$$

where $f_k(x) \in \mathcal{H}_K$; $\mathcal{H}_K$ is the reproducing kernel Hilbert space generated by a positive definite kernel $K(\cdot,\cdot)$. Notice that $f_1(x), \ldots, f_K(x)$ are not identifiable in this model, for if we add a common term to each $f_k(x)$, $p_1(x), \ldots, p_K(x)$ will not change. To make $f_k(x)$ identifiable, we consider the symmetric constraint

$$\sum_{k=1}^{K} f_k(x) = 0. \tag{2.43}$$

Then the multi-class kernel logistic regression fits a model to minimize the regularized

negative log-likelihood

$$H = -\sum_{i=1}^{n} \ln p_{y_i}(x_i) + \frac{\lambda}{2}\|f\|^2_{\mathcal{H}_K} \tag{2.44}$$

$$= \sum_{i=1}^{n}\left[-y_i^T f(x_i) + \ln\left(e^{f_1(x_i)} + \cdots + e^{f_K(x_i)}\right)\right] + \frac{\lambda}{2}\|f\|^2_{\mathcal{H}_K} \tag{2.45}$$

where y_i is a binary K-vector with values all zero except a 1 in position k if the class is k, and

$$f(x_i) = (f_1(x_i), \ldots, f_K(x_i))^T, \tag{2.46}$$

$$\|f\|^2_{\mathcal{H}_K} = \sum_{k=1}^{K} \|f_k\|^2_{\mathcal{H}_K}. \tag{2.47}$$

Using the representer theorem (Kimeldorf & Wahba (1971)), one can show that $f_k(x)$, which minimizes H, has the form

$$f_k(x) = \beta_{0k} + \sum_{i=1}^{n} \alpha_{ik} K(x_i, x). \tag{2.48}$$

Hence, (2.44) becomes

$$H = \sum_{i=1}^{n}\left[-y_i^T (K_1(i,)A)^T + \ln\left(1^T e^{(K_1(i,)A)^T}\right)\right] + \frac{\lambda}{2}\sum_{k=1}^{K} \alpha_k^T K_2 \alpha_k \tag{2.49}$$

where $A = (\alpha_1 \ldots \alpha_K)$, K_1 and K_2 are defined in the same way as in the two-class case; and $K_1(i,)$ is the ith row of K_1. Notice that in this model, the constraint (2.43) is not necessary anymore, for at the minimum of (2.44), $\sum_{k=1}^{K} f_k(x) = 0$ is automatically satisfied.

2.3.1 Multi-class KLR and Multi-class SVM

Similar to Theorem 2.1, a connection between the multi-class kernel logistic regression and the multi-class support vector machine also exists. Let

$$\mathcal{D} = \{h_1(x), \ldots, h_q(x)\}$$

be the dictionary of the basis functions of the enlarged feature space. Consider

$$\min_{\beta_{0k},\beta_k} \quad \sum_{i=1}^{n} \left[-y_i^T f(x_i) + \ln \left(e^{f_1(x_i)} + \cdots + e^{f_K(x_i)} \right) \right] \tag{2.50}$$

$$\text{subject to} \quad f_k(x_i) = \beta_{0k} + h(x_i)^T \beta_k, \ i = 1, \ldots, n \tag{2.51}$$

$$\sum_{k=1}^{K} \|\beta_k\|_2^2 \leq s. \tag{2.52}$$

Theorem 2.2 *Suppose the training data are pairwise separable, i.e.* $\exists \beta_{0k}, \beta_k$, *s.t.* $(\beta_{0y_i} - \beta_{0k}) + h(x_i)^T(\beta_{y_i} - \beta_k) > 0, \ \forall i, \ \forall k \neq y_i$. *Let the solution of (2.50)-(2.52) be denoted by* $\hat{\beta}_k(s)$, *then*

$$\frac{\hat{\beta}_k(s)}{s} \to \beta^* \quad as \quad s \to \infty,$$

where β^* *is the solution of the multi-class support vector machine (1.18)-(1.22), if* β^* *is unique.*

If β^* *is not unique, then* $\frac{\hat{\beta}_k(s)}{s}$ *may have multiple convergence points, but they are all solutions of (1.18)-(1.22).*

The proof of the theorem is very similar to that of Theorem 2.1, we omit it here.

2.3.2 Multi-class IVM

The multi-class import vector machine procedure is similar to the two-class case, and the computational cost is $O(Kn^2m^2)$. Figure 2.7 is a simulation of the multi-class import vector machine. The data in each class are generated from a mixture of Gaussians (Hastie,

Tibshirani & Friedman (2001)).

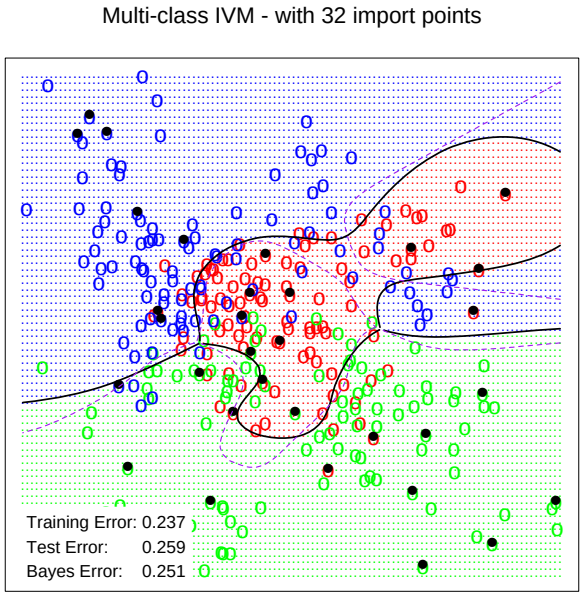

Figure 2.7: *Radial kernel is used.* $K = 3$, $N = 300$, $\lambda = 0.368$, $|\mathcal{S}| = 32$.

2.4 Summary

We have discussed the import vector machine method in two-class and multi-class classification. We showed that it not only performs as well as the support vector machine, but also provides an estimate of the probability $p(x)$. The computational cost of the import vector machine is $O(n^2m^2)$ for the two-class case and $O(Kn^2m^2)$ for the multi-class case, where m is the number of import points.

Chapter 3

1-norm Support Vector Machines

In Chapter 2, we replace the hinge loss function of the support vector machine with the binomial log-likelihood and fit kernel logistic regression and import vector machine models. In this chapter, we replace the L_2-norm penalty of the support vector machine with the L_1-norm, and consider the 1-norm support vector machine. We argue that the 1-norm support vector machine may have some advantage over the standard 2-norm support vector machine, especially when there are redundant noise features. We also propose an efficient algorithm that computes the whole solution path of the 1-norm support vector machine, hence facilitates adaptive selection of the tuning parameter for the 1-norm support vector machine.

3.1 Introduction

In standard two-class classification problems, we are given a set of training data (x_1, y_1), $\ldots (x_n, y_n)$, where the input $x_i \in \mathcal{R}^p$, and the output $y_i \in \{1, -1\}$ is binary. We wish to find a classfication rule from the training data, so that when given a new input x, we can assign a class y from $\{1, -1\}$ to it.

To handle this problem, we consider the 1-norm support vector machine:

$$\min_{\beta_0,\beta} \quad \sum_{i=1}^{n}\left[1-y_i\left(\beta_0+\sum_{j=1}^{q}\beta_j h_j(x_i)\right)\right]_+ \tag{3.1}$$

$$\text{s.t.} \quad \|\beta\|_1 = |\beta_1|+\cdots+|\beta_q| \le s, \tag{3.2}$$

where

$$\mathcal{D} = \{h_1(x), \ldots h_q(x)\}$$

is a dictionary of basis functions, and s is a tuning parameter. The solution is denoted as $\hat{\beta}_0(s)$ and $\hat{\beta}(s)$; the fitted model is

$$\hat{f}(x) = \hat{\beta}_0 + \sum_{j=1}^{q}\hat{\beta}_j h_j(x). \tag{3.3}$$

The classification rule is given by $sign[\hat{f}(x)]$. The 1-norm support vector machine has been successfully used in Bradley & Mangasarian (1998) and Song, Breneman, Bi, Sukumar, Bennett, Cramer & Tugcu (2002). We argue in this chapter that the 1-norm support vector machine may have some advantage over the standard 2-norm support vector machine, especially when there are redundant noise features.

To get a good fitted model $\hat{f}(x)$ that performs well on future data, we also need to select an appropriate tuning parameter s. In practice, people usually pre-specify a finite set of values for s that covers a wide range, then either use a separate validation data set or use cross-validation to select a value for s that gives the best performance among the given set. In this paper, we illustrate that the solution path $\hat{\beta}(s)$ is piece-wise linear as a function of s (in the $\mathcal{R}^q$ space); we also propose an efficient algorithm to compute the exact whole solution path

$$\{\hat{\beta}(s), 0 \le s \le \infty\},$$

hence help us understand how the solution changes with s and facilitate the adaptive selection of the tuning parameter s. Under some mild assumptions, we show that the computational cost to compute the whole solution path $\hat{\beta}(s)$ is $O(nq \min(n,q)^2)$ in the worst case and $O(nq)$ in the best case.

Before delving into the technical details, we illustrate the concept of piece-wise linearity of the solution path $\hat{\beta}(s)$ with a simple example. We generate 10 training data in each of two classes. The first class has two standard normal independent inputs x_1, x_2. The second class also has two standard normal independent inputs, but conditioned on $4.5 \leq x_1^2 + x_2^2 \leq 8$. The dictionary of basis functions is

$$\mathcal{D} = \{\sqrt{2}x_1, \sqrt{2}x_2, \sqrt{2}x_1x_2, x_1^2, x_2^2\}.$$

The solution path $\hat{\beta}(s)$ as a function of s is shown in Figure 3.1. Any segment between two adjacent vertical lines is linear. Hence the right derivative of $\hat{\beta}(s)$ with respect to s is piece-wise constant (in $\mathcal{R}^q$). The two solid paths are for x_1^2 and x_2^2, which are the two relevant features.

In section 3.2, we motivate why we are interested in the 1-norm support vector machine. In section 3.3, we describe the algorithm that computes the whole solution path $\hat{\beta}(s)$. In section **??**, we show some numerical results on both simulation data and real world data.

3.2 Regularized support vector machines

The standard 2-norm support vector machine is equivalent to fit a model that

$$\min_{\beta_0,\beta_j} \sum_{i=1}^{n} \left[1 - y_i \left(\beta_0 + \sum_{j=1}^{q} \beta_j h_j(x_i)\right)\right]_+ + \lambda\|\beta\|_2^2, \tag{3.4}$$

where λ is a tuning parameter. In practice, people usually choose $h_j(x)$'s to be the basis functions of a reproducing kernel Hilbert space. Then a kernel trick allows the dimension of the transformed feature space to be very large, even infinite in some cases (i.e. $q = \infty$),

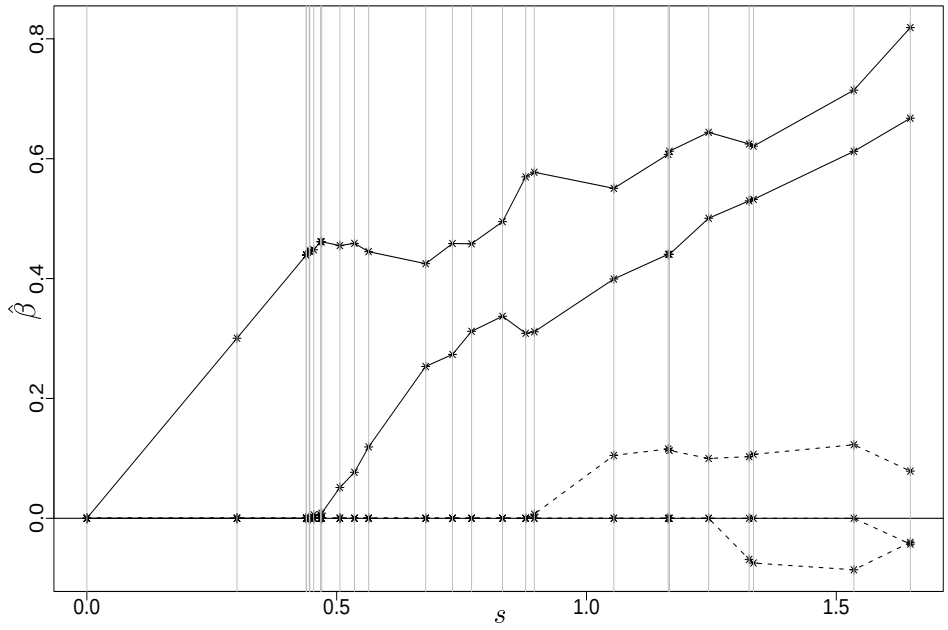

Figure 3.1: The solution path $\hat{\beta}(s)$ as a function of s.

without causing extra computational burden, see Evgeniou, Pontil & Poggio (1999) and Wahba (1999). In this chapter, however, we will concentrate on the basis representation (3.3) rather than a kernel representation.

Notice that (3.4) has the form $loss+penalty$, and λ is the tuning parameter that controls the tradeoff between loss and penalty. The loss $(1-yf)_+$ is called the *hinge* loss, and the penalty is called the *ridge* penalty. The idea of penalizing by the sum-of-squares of the parameters is also used in neural networks, where it is known as *weight decay*. The ridge penalty shrinks the fitted coefficients $\hat{\beta}$ towards zero. It is well known that this shrinkage has the effect of controlling the variances of $\hat{\beta}$, hence possibly improves the fitted model's prediction accuracy, especially when there are many highly correlated features, see Hastie, Tibshirani & Friedman (2001). So from a statistical function estimation point of view, the ridge penalty could possibly explain the success of the support vector machine (Hastie, Tibshirani & Friedman (2001) and Wahba (1999)). On the other hand, computational learning theory has associated the good performance of the support vector machine to its

margin maximizing property (Vapnik (1995)), a property of the hinge loss. Rosset, Zhu & Hastie (2003) makes some effort to build a connection between these two different views.

In this chapter, we replace the ridge penalty in (3.4) with the L_1-norm of β, i.e. the *lasso* penalty (Tibshirani (1996)), and consider the 1-norm support vector machine problem:

$$\min_{\beta_0,\beta} \sum_{i=1}^{n} \left[1 - y_i \left(\beta_0 + \sum_{j=1}^{q} \beta_j h_j(x_i) \right) \right]_+ + \lambda \|\beta\|_1, \tag{3.5}$$

which is an equivalent Lagrange version of the optimization problem (3.1)-(3.2).

The lasso penalty was first proposed in Tibshirani (1996) for regression problems, where the response y is continuous rather than categorical. It has also been used in Bradley & Mangasarian (1998) and Song, Breneman, Bi, Sukumar, Bennett, Cramer & Tugcu (2002) for classification problems under the framework of support vector machines. Similar to the ridge penalty, the lasso penalty also shrinks the fitted coefficients $\hat{\beta}$'s towards zero, hence (3.5) also benefits from the reduction in fitted coefficients' variances. Another property of the lasso penalty is that because of the L_1 nature of the penalty, making λ sufficiently large, or equivalently s sufficiently small, will cause some of the coefficients $\hat{\beta}_j$'s to be *exactly zero*. For example, when $s = 1$ in Figure 3.1, only three fitted coefficients are non-zero. Thus the lasso penalty does a kind of continuous feature selection, while this is not the case for the ridge penalty. In (3.4), none of the $\hat{\beta}_j$'s will be equal to zero.

It is interesting to note that the ridge penalty corresponds to a Gaussian prior for the β_j's, while the lasso penalty corresponds to a double-exponential prior. The double-exponential density has heavier tails than the Gaussian density. This reflects the greater tendency of the lasso to produce some large fitted coefficients and leave others at 0, especially in high dimensional problems. Recently, Friedman, Hastie, Rosset, Tibshirani & Zhu (2004) consider a situation where we have a small number of training data, e.g. $n = 100$, and a large number of basis functions, e.g. $q = 10,000$. Friedman, Hastie, Rosset, Tibshirani & Zhu (2004) argue that in the *sparse* scenario, i.e. only a small number of true coefficients

β_j's are non-zero, the lasso penalty works better than the ridge penalty; while in the non-sparse scenario, e.g. the true coefficients β_j's have a Gaussian distribution, neither the lasso penalty nor the ridge penalty will fit the coefficients well, since there is too little data from which to estimate these non-zero coefficients. This is the *curse of dimensionality* taking its toll. Based on these observations, Friedman, Hastie, Rosset, Tibshirani & Zhu (2004) further propose the *bet on sparsity* principle for high-dimensional problems, which encourages using lasso penalty.

3.3 Algorithm

Section 3.2 gives the motivation why we are interested in the 1-norm support vector machine. To solve the 1-norm support vector machine for a *fixed* value of s, we can transform (3.1)-(3.2) into a linear programming problem and use standard software packages; but to get a good fitted model $\hat{f}(x)$ that performs well on future data, we need to select an appropriate value for the tuning paramter s. In this section, we propose an efficient algorithm that computes the whole solution path $\hat{\beta}(s)$, hence facilitates adaptive selection of s.

3.3.1 Piece-wise linearity

If we follow the solution path $\hat{\beta}(s)$ of (3.1)-(3.2) as s increases, we will notice that since both $\sum_i (1 - y_i \hat{f}_i)_+$ and $\|\beta\|_1$ are piece-wise linear, the Karush-Kuhn-Tucker conditions will not change when s increases unless a *residual* $(1 - y_i \hat{f}_i)$ changes from non-zero to zero, or a fitted coefficient $\hat{\beta}_j(s)$ changes from non-zero to zero, which correspond to the non-smooth points of $\sum_i (1 - y_i \hat{f}_i)_+$ and $\|\beta\|_1$. This implies that the derivative of $\hat{\beta}(s)$ with respect to s is piece-wise constant, because when the Karush-Kuhn-Tucker conditions do not change, the derivative of $\hat{\beta}(s)$ will not change either. Hence it indicates that the whole solution path $\hat{\beta}(s)$ is piece-wise linear. See Appendix for details.

Thus to compute the whole solution path $\hat{\beta}(s)$, all we need to do is to find the *joints*, i.e. the asterisk points in Figure 3.1, on this piece-wise linear path, then use straight lines to interpolate them, or equivalently, to start at $\hat{\beta}(0) = 0$, find the right derivative of $\hat{\beta}(s)$,

let s increase and only change the derivative when $\hat{\beta}(s)$ gets to a joint.

3.3.2 Initial solution (i.e. $s = 0$)

The following notation is used. Let

$$\mathcal{V} = \{j : \hat{\beta}_j(s) \neq 0\}, \tag{3.6}$$

$$\mathcal{E} = \{i : 1 - y_i \hat{f}_i = 0\}, \tag{3.7}$$

$$\mathcal{L} = \{i : 1 - y_i \hat{f}_i > 0\}, \tag{3.8}$$

and u for the right derivative of $\hat{\beta}_{\mathcal{V}}(s)$: $\|u\|_1 = 1$ and $\hat{\beta}_{\mathcal{V}}(s)$ denotes the components of $\hat{\beta}(s)$ with indices in $\mathcal{V}$. Without loss of generality, we assume $\#\{y_i = 1\} \geq \#\{y_i = -1\}$; then $\hat{\beta}_0(0) = 1, \hat{\beta}_j(0) = 0$. To compute the path that $\hat{\beta}(s)$ follows, we need to compute the derivative of $\hat{\beta}(s)$ at 0. We consider a modified problem:

$$\min_{\beta_0, \beta_j} \quad \sum_{y_i=1} (1 - y_i f_i)_+ + \sum_{y_i=-1} (1 - y_i f_i) \tag{3.9}$$

$$\text{s.t.} \quad \|\beta\|_1 \leq \Delta s, \quad f_i = \beta_0 + \sum_{j=1}^{q} \beta_j h_j(x_i). \tag{3.10}$$

Notice that if $y_i = 1$, the loss is still $(1 - y_i f_i)_+$; but if $y_i = -1$, the loss becomes $(1 - y_i f_i)$. In this setup, the derivative of $\hat{\beta}(\Delta s)$ with respect to Δs is the same no matter what value Δs is, and one can show that it coincides with the right derivative of $\hat{\beta}(s)$ when s is sufficiently small. Hence this setup helps us find the initial derivative u of $\hat{\beta}(s)$. Solving (3.9)-(3.10), which can be transformed into a simple linear programming problem, we get initial $\mathcal{V}$, $\mathcal{E}$ and $\mathcal{L}$. $|\mathcal{V}|$ should be equal to $|\mathcal{E}|$. We also have:

$$\begin{pmatrix} \hat{\beta}_0(\Delta s) \\ \hat{\beta}_{\mathcal{V}}(\Delta s) \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix} + \Delta s \cdot \begin{pmatrix} u_0 \\ u \end{pmatrix}. \tag{3.11}$$

Δs starts at 0 and increases.

3.3.3 Main algorithm

The main algorithm that computes the whole solution path $\hat{\beta}(s)$ proceeds as following:

1. Increase Δs until one of the following two events happens:

   - A training point hits $\mathcal{E}$, i.e. $1 - y_i f_i \neq 0$ becomes $1 - y_i f_i = 0$ for some i.
   - A basis function in $\mathcal{V}$ leaves $\mathcal{V}$, i.e. $\hat{\beta}_j \neq 0$ becomes $\hat{\beta}_j = 0$ for some j.

   Let the current $\hat{\beta}_0$, $\hat{\beta}$ and s be denoted by $\hat{\beta}_0^{old}$, $\hat{\beta}^{old}$ and s^{old}.

2. For each $j^* \notin \mathcal{V}$, we solve:

$$\begin{cases} u_0 + \sum_{\mathcal{V}} u_j h_j(x_i) + u_{j^*} h_{j^*}(x_i) &= 0 \quad \text{for } i \in \mathcal{E} \\ \sum_{\mathcal{V}} sign(\hat{\beta}_j^{old}) u_j + |u_{j^*}| &= 1 \end{cases} \tag{3.12}$$

   where u_0, u_j and u_{j^*} are the unknowns. We then compute:

$$\frac{\Delta loss_{j^*}}{\Delta s} = \sum_{\mathcal{L}} y_i \left(u_0 + \sum_{\mathcal{V}} u_j h_j(x_i) + u_{j^*} h_{j^*}(x_i) \right). \tag{3.13}$$

3. For each $i' \in \mathcal{E}$, we solve:

$$\begin{cases} u_0 + \sum_{\mathcal{V}} u_j h_j(x_i) &= 0 \quad \text{for } i \in \mathcal{E} \backslash \{i'\} \\ \sum_{\mathcal{V}} sign(\hat{\beta}_j^{old}) u_j &= 1 \end{cases} \tag{3.14}$$

   where u_0 and u_j are the unknowns. We then compute:

$$\frac{\Delta loss_{i'}}{\Delta s} = \sum_{\mathcal{L}} y_i \left(u_0 + \sum_{\mathcal{V}} u_j h_j(x_i) \right). \tag{3.15}$$

4. Compare the computed values of $\frac{\Delta loss}{\Delta s}$ from step 2 and step 3. There are $q - |\mathcal{V}| + |\mathcal{E}| = q + 1$ such values. Choose the smallest negative $\frac{\Delta loss}{\Delta s}$. Hence,

   - If the smallest $\frac{\Delta loss}{\Delta s}$ is non-negative, the algorithm terminates; else

- If the smallest negative $\frac{\Delta loss}{\Delta s}$ corresponds to a j^* in step 2, we update

$$\mathcal{V} \leftarrow \mathcal{V} \cup \{j^*\}, \tag{3.16}$$

$$u \leftarrow \begin{pmatrix} u \\ u_{j^*} \end{pmatrix}. \tag{3.17}$$

- If the smallest negative $\frac{\Delta loss}{\Delta s}$ corresponds to a i' in step 3, we update u and

$$\mathcal{E} \leftarrow \mathcal{E} \backslash \{i'\}, \tag{3.18}$$

$$\mathcal{L} \leftarrow \mathcal{L} \cup \{i'\} \text{ if necessary.} \tag{3.19}$$

In either of the last two cases, $\hat{\beta}(s)$ changes as:

$$\begin{pmatrix} \hat{\beta}_0(s^{old} + \Delta s) \\ \hat{\beta}_{\mathcal{V}}(s^{old} + \Delta s) \end{pmatrix} = \begin{pmatrix} \hat{\beta}_0^{old} \\ \hat{\beta}_{\mathcal{V}}^{old} \end{pmatrix} + \Delta s \cdot \begin{pmatrix} u_0 \\ u \end{pmatrix}, \tag{3.20}$$

and we go back to step 1.

In the end, we get a path $\hat{\beta}(s)$, which is piece-wise linear.

3.3.4 Remarks

We postpone the proof that this algorithm does indeed give the exact whole solution path $\hat{\beta}(s)$ of (3.1)-(3.2) to the Appendix. Here instead, we explain a little what each step of the algorithm tries to do.

Step 1 of the algorithm indicates that $\hat{\beta}(s)$ gets to a joint on the solution path and the right derivative of $\hat{\beta}(s)$ needs to be changed if either a residual $(1 - y_i \hat{f}_i)$ changes from non-zero to zero, or the coefficient of a basis function $\hat{\beta}_j(s)$ changes from non-zero to zero, when s increases. Then there are two possible types of actions that the algorithm can take: (1) add a basis function into $\mathcal{V}$, or (2) remove a point from $\mathcal{E}$.

Step 2 computes the possible right derivative of $\hat{\beta}(s)$ if adding each basis function $h_{j^*}(x)$ into $\mathcal{V}$. Step 3 computes the possible right derivative of $\hat{\beta}(s)$ if removing each point i' from

$\mathcal{E}$. The possible right derivative of $\hat{\beta}(s)$ (determined by either (3.12) or (3.14)) is such that the training points in $\mathcal{E}$ are kept in $\mathcal{E}$ when s increases, until the next joint (step 1) occurs. $\Delta loss/\Delta s$ indicates how fast the $loss$ will decrease if $\hat{\beta}(s)$ changes according to u. Step 4 takes the action corresponding to the smallest negative $\Delta loss/\Delta s$. When the $loss$ can not be decreased, the algorithm terminates.

3.3.5 Computational cost

We have proposed an algorithm that computes the whole solution path $\hat{\beta}(s)$. A natural question is then what is the computational cost of this algorithm? Suppose $|\mathcal{E}| = m$ at a joint on the piece-wise linear solution path, then it takes $O(qm^2)$ to compute step 2 and step 3 of the algorithm through Sherman-Morrison updating formula. If we assume the training data are separable by the dictionary $\mathcal{D}$, then all the training data are eventually going to have loss $(1 - y_i\hat{f}_i)_+$ equal to zero. Hence it is reasonable to assume the number of joints on the piece-wise linear solution path is $O(n)$. Since the maximum value of m is $\min(n, q)$ and the minimum value of m is 1, we get the worst computational cost is $O(nq\min(n, q)^2)$ and the best computational cost is $O(nq)$. Notice that this is a rough calculation of the computational cost under some mild assumptions. Simulation results (section **??**) actually indicate that the number of joints tends to be $O(\min(n, q))$.

3.4 Numerical results

In this section, we use both simulation and real data results to illustrate the 1-norm support vector machine.

3.4.1 Simulation results

The data generation mechanism is the same as the one described in section 3.1, except that we generate 50 training data in each of two classes, and to make harder problems, we sequentially augment the inputs with additional two, four, six and eight standard normal noise inputs. Hence the second class almost completely surrounds the first, like the skin

Table 3.1: Simulation results of 1-norm and 2-norm support vector machine

		Test Error (SE)				
	Simulation	**1-norm**	**2-norm**	**No Penalty**	$\lvert\mathcal{D}\rvert$	**# Joints**
1	No noise input	0.073 (0.010)	0.08 (0.02)	0.08 (0.01)	5	94 (13)
2	2 noise inputs	0.074 (0.014)	0.10 (0.02)	0.12 (0.03)	14	149 (20)
3	4 noise inputs	0.074 (0.009)	0.13 (0.03)	0.20 (0.05)	27	225 (30)
4	6 noise inputs	0.082 (0.009)	0.15 (0.03)	0.22 (0.06)	44	374 (52)
5	8 noise inputs	0.084 (0.011)	0.18 (0.03)	0.22 (0.06)	65	499 (67)

surrounding the oragne, in a two-dimensional subspace. The Bayes error rate for this problem is 0.0435, irrespective of dimension. In the original input space, a hyperplane cannot separate the classes; we use an enlarged feature space corresponding to the 2nd degree polynomial kernel, hence the dictionary of basis functions is

$$\mathcal{D} = \{\sqrt{2}x_j, \sqrt{2}x_j x_{j'}, x_j^2, j, j' = 1, \ldots p\}.$$

We generate 1000 test data to compare the 1-norm support vector machine and the standard 2-norm support vector machine. The average test errors over 50 simulations, with different numbers of noise inputs, are shown in Table 3.1. For both the 1-norm support vector machine and the 2-norm support vector machine, we choose the tuning parameters to minimize the test error, to be as fair as possible to each method. For comparison, we also include the results for the non-penalized support vector machine.

From Table 3.1 we can see that the non-penalized support vector machine performs significantly worse than the penalized ones; the 1-norm support vector machine and the 2-norm support vector machine perform similarly when there is no noise input (line 1), but the 2-norm support vector machine is adversely affected by noise inputs (line 2 - line 5). Since the 1-norm support vector machine has the ability to select relevant features and ignore redundant features, it does not suffer from the noise inputs as much as the 2-norm support vector machine. Table 3.1 also shows the number of basis functions q and the number of joints on the piece-wise linear solution path. Notice that $q < n$ and there is a

striking linear relationship between $|\mathcal{D}|$ and $\#Joints$ (Figure 3.2). Figure 3.2 also shows the 1-norm support vector machine result for one simulation.

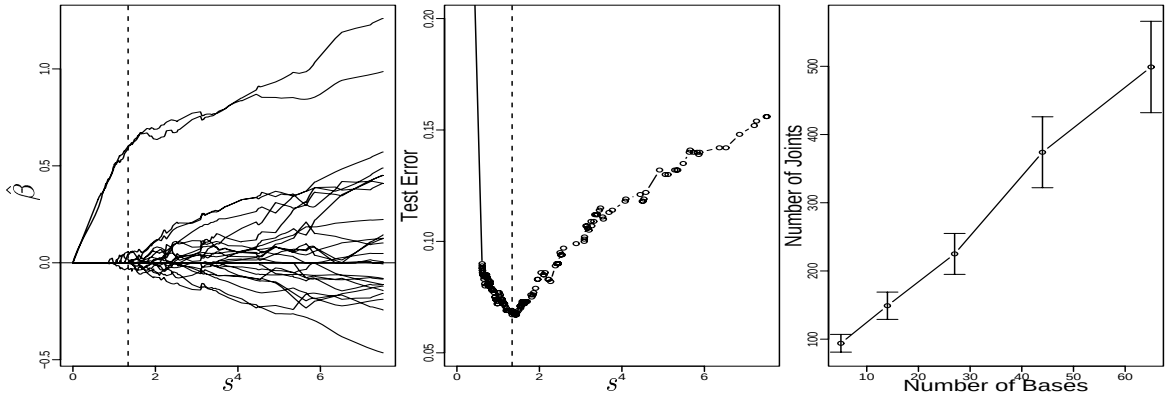

Figure 3.2: Left and middle panels: 1-norm support vector machine when there are 4 noise inputs. The left panel is the piece-wise linear solution path $\hat{\beta}(s)$. The two upper paths correspond to x_1^2 and x_2^2, which are the relevant features. The middle panel is the test error along the solution path. The dash lines correspond to the minimum of the test error. The right panel illustrates the linear relationship between the number of basis functions and the number of joints on the solution path when $q < n$.

3.4.2 Real data results

In this section, we apply the 1-norm support vector machine to classification of gene microarrays. Detailed description on classification of gene microarrays are in Chapter 4. Classification of patient samples is an important aspect of cancer diagnosis and treatment. The 2-norm support vector machine has been successfully applied to microarray cancer diagnosis problems (Guyon, Weston, Barnhill & Vapnik (2002) and Mukherjee, Tamayo, Slonim, Verri, Golub, Mesirov & Poggio (1999)). However, one weakness of the 2-norm support vector machine is that it only predicts a cancer class label but does not automatically select relevant genes for the classification. Often a primary goal in microarray cancer diagnosis is to identify the genes responsible for the classification, rather than class prediction. Golub, Slonim, Tamayo, Huard, Gaasenbeek, Mesirov, Coller, Loh, Downing & Caligiuri (2000) and Guyon, Weston, Barnhill & Vapnik (2002) have proposed gene selection methods, which we call univariate ranking (UR) and recursive feature elimination (RFE) (see [14]), that can be

Table 3.2: Results on Microarray Classification

Method	CV Error	Test Error	# of Genes
2-norm SVM UR	2/38	3/34	22
2-norm SVM RFE	2/38	1/34	31
1-norm SVM	2/38	2/34	17

combined with the 2-norm support vector machine. However, these procedures are two-step procedures that depend on external gene selection methods. On the other hand, the 1-norm support vector machine has an inherent gene (feature) selection property due to the lasso penalty. Hence the 1-norm support vector machine achieves the goals of classification of patients and selection of genes simultaneously.

We apply the 1-norm support vector machine to leukemia data Golub, Slonim, Tamayo, Huard, Gaasenbeek, Mesirov, Coller, Loh, Downing & Caligiuri (2000). This data set consists of 38 training data and 34 test data of two types of acute leukemia, *acute myeloid leukemia* (AML) and *acute lymphoblastic leukemia* (ALL). Each datum is a vector of $p = 7,129$ genes. We use the original input x_j, i.e. the jth gene's expression level, as the basis function, i.e. $q = p$. The tuning parameter is chosen according to 10-fold cross-validation, then the final model is fitted on all the training data and evaluated on the test data. The number of joints on the solution path is 104, which appears to be $O(n) \ll O(q)$. The results are summarized in Table 3.2. We can see that the 1-norm support vector machine performs similarly to the other methods in classification and it has the advantage of automatically selecting relevant genes. We should notice that the maximum number of genes that the 1-norm support vector machine can select is upper bounded by n, which is usually much less than q in microarray problems.

3.5 Summary

We have considered the 1-norm support vector machine in this chapter. We illustrate that the 1-norm support vector machine may have some advantage over the 2-norm support

vector machine, especially when there are redundant features. The solution path $\hat{\beta}(s)$ of the 1-norm support vector machine is a piece-wise linear function in the tuning parameter s. We have proposed an efficient algorithm to compute the whole solution path $\hat{\beta}(s)$ of the 1-norm support vector machine, and facilitate adaptive selection of the tuning parameter s.

Chapter 4

Microarray Classification

Classification of patient samples is an important aspect of cancer diagnosis and treatment. The support vector machine has been successfully applied to microarray cancer diagnosis problems. However, one weakness of the support vector machine is that given a tumor sample, it only predicts a cancer class label but does not provide any estimate of the underlying probability. We propose penalized logistic regression (PLR) as an alternative to the support vector machine for the microarray cancer diagnosis problem. We show that when using the same set of genes, penalized logistic regression and the support vector machine perform similarly in cancer classification, but penalized logistic regression has the advantage of additionally providing an estimate of the underlying probability. Often a primary goal in microarray cancer diagnosis is to identify the genes responsible for the classification, rather than class prediction. We consider two gene selection methods in this chapter, univariate ranking (UR) and recursive feature elimination (RFE), that can be combined with the support vector machine and penalized logistic regression. Empirical results indicate that penalized logistic regression combined with recursive feature elimination tends to select less genes than other methods and also performs well in both cross-validation and test samples.

4.1 Introduction

Classification of patient samples is an important aspect of cancer diagnosis and treatment. Conventional diagnosis of cancer has been based on examination of the morphological appearance of stained tissue specimens under light microscopy. However, this method is subjective and depends on highly trained pathologists. Microarrays offer hope that cancer classification can be objective and highly accurate, providing clinicians with the information to choose the most appropriate forms of treatment. See Golub, Slonim, Tamayo, Huard, Gaasenbeek, Mesirov, Coller, Loh, Downing & Caligiuri (2000), Khan, Wei, Ringner, Saal, Ladanyi, Westermann, Berthold, Schwab, Antonescu & Peterson (2001), Lee & Lee (2003), Ramaswamy, Tamayo, Rifkin, Mukherjee, Yeang, Angelo, Ladd, Reich, Latulippe, Mesirov, Poggio, Gerald, Loda, Lander & Golub (2002), Tibshirani, Hastie, Narasimhan & Chu (2002) and references therein for recent work.

The support vector machine (SVM) is one of the methods that has been successfully applied to the cancer diagnosis problem in previous studies (Lee & Lee (2003), Mukherjee, Tamayo, Slonim, Verri, Golub, Mesirov & Poggio (1999), Ramaswamy, Tamayo, Rifkin, Mukherjee, Yeang, Angelo, Ladd, Reich, Latulippe, Mesirov, Poggio, Gerald, Loda, Lander & Golub (2002)). In two-class classification, the linear support vector machine fits a model $f(x) = \beta_0 + \sum_{j=1}^{p} \beta_j x_j$ that minimizes

$$\sum_{i=1}^{n} [1 - y_i f(x_i)]_+ + \frac{\lambda}{2}(\|\beta_1\|^2 + \cdots \|\beta_p\|^2). \tag{4.1}$$

The classification decision is then made according to $sign[f(x)]$. However, one weakness of the support vector machine is that it only estimates $sign[p(x) - 1/2]$, while the probability $p(x)$ is often of interest itself, where $p(x) = P(C = 1|X = x)$ is the conditional probability of a point being in class 1 given $X = x$. In going from two-class to multi-class classification, the one-vs-rest scheme is often used: given K classes, the problem is divided into a series of K one-vs-rest problems, and each one-vs-rest problem is addressed by a different class-specific support vector machine classifier (e.g., "class 1" vs. "not class 1"); then a new sample takes

the class of the classifier with the largest real valued output $c = \text{argmax}_{k=1,...K} f_k$, where f_k is the real valued output of the kth support vector machine classifier. Recently, Lee & Lee (2003) extends the two-class support vector machine to multi-class case in a more direct way and estimates $argmax_k P(C = k|X = x)$ directly, but it still lacks the estimates of $P(C = k|X = x)$ themselves.

In this chapter, we use penalized logistic regression (PLR) classifier to address the cancer diagnosis problem. The motivation for the use of penalized logistic regression is pointed out in Chapter 2. Penalized logistic regression not only performs as well as the support vector machine in two-class classification, but can also naturally be generalized to the multi-class case. Furthermore, penalized logistic regression provides an estimate of the underlying class probabilities $p(x)$.

Maximum likelihood and the Newton-Raphson algorithm is the traditional way to solve penalized logistic regression numerically. However, the computation is prohibitive when the number of variables is large. We use a transformation trick to make the computation feasible, but the computational cost can still be high. To further reduce the computation, we use a sequential minimal optimization (SMO) algorithm (Platt (1999)) to solve penalized logistic regression in this paper. This method was first proposed in Keerthi, Duan, Shevade & Poo (2002) for two-class classification; we generalize it to the multi-class case.

Besides predicting the correct cancer class for a given tumor sample, another challenge in microarray cancer diagnosis is to identify the relevant genes which contribute most to the classification. We consider two feature (gene) selection methods in this paper, univariate ranking (UR) (Dudoit, Fridlyand & Speed (2002), Golub, Slonim, Tamayo, Huard, Gaasenbeek, Mesirov, Coller, Loh, Downing & Caligiuri (2000)), and recursive feature elimination (RFE) (Guyon, Weston, Barnhill & Vapnik (2002)), and compare their performance when used with both penalized logistic regression and the support vector machine on three cancer diagnosis data sets.

Our analysis of these data sets indicates that penalized logistic regression and the support vector machine perform similarly when combined with either univariate ranking or recursive feature elimination; the recursive feature elimination method seems to perform

better than the univariate ranking method; penalized logistic regression tends to select less genes than the support vector machine. Overall, penalized logistic regression with recursive feature elimination seems to perform the best in both predicting the cancer class and reducing the redundant genes.

The formulation of penalized logistic regression is given in section 4.2. In section 4.3, we describe the two gene selection methods, univariate ranking and recursive feature elimination. In section 4.4, we apply penalized logistic regression and the support vector machine to three microarray cancer data sets.

4.2 Penalized Logistic Regression

In standard K-class classification problems, we are given a set of training data (x_1, c_1), (x_2, c_2), $\ldots (x_n, c_n)$, where the input x is a p-vector $x_i = (x_{i1}, x_{i2}, \ldots x_{ip})^T$, the output c_i is qualitative and assumes values in a finite set $\{1, 2, \ldots K\}$. We wish to find a classfication rule from the training data, so that when given a new input x, we can assign a class label k from $\{1, 2, \ldots K\}$ to it. Usually it is assumed that the training data are an independently and identically distributed sample from an unknown probability distribution $P(X, C)$. The conditional probability of a point being in class k given $X = x$ is denoted as $p_k(x) = P(C = k|X = x)$. The Bayes classification rule is given by:

$$\text{argmax}_{k \in \{1,2,\ldots K\}} p_k(x)$$

4.2.1 Penalized logistic regression Formulation

The logistic regression model arises from the desire to model the conditional probabilities of the K classes via linear functions in x, while at the same time ensuring that they sum

to one and remain in $[0, 1]$. The model has the form

$$\begin{aligned} p_1(x) &= \frac{e^{f_1(x)}}{\sum_{k=1}^{K} e^{f_k(x)}}, \\ p_2(x) &= \frac{e^{f_2(x)}}{\sum_{k=1}^{K} e^{f_k(x)}}, \\ &\vdots \\ p_K(x) &= \frac{e^{f_K(x)}}{\sum_{k=1}^{K} e^{f_k(x)}}, \end{aligned}$$

where

$$f_k(x) = \beta_{k0} + \sum_{j=1}^{p} \beta_{kj} x_j = \beta_{k0} + \beta_k^T x, \quad k = 1, 2, \ldots K. \tag{4.2}$$

Notice that $f_1(x), f_2(x), \ldots f_K(x)$ are unidentifiable in this model, for if we add a common $\beta_0 + \sum_{j=1}^{p} \beta_j x_j$ to each $f_k(x)$, $p_1(x), p_2(x), \ldots p_K(x)$ will not change. To make $f_k(x)$ identifiable, we consider the symmetric constraint $\sum_{k=1}^{K} f_k(x) = 0$, or more explicitly

$$\sum_{k=1}^{K} \beta_{k0} = 0 \tag{4.3}$$

$$\sum_{k=1}^{K} \beta_k = 0. \tag{4.4}$$

Logistic regression models are usually fit by maximum likelihood. Since $p_k(x)$ completely specifies the conditional probabilities, the multinomial distribution is appropriate. Given the training set (x_1, c_1), (x_2, c_2), $\ldots (x_n, c_n)$, the negative multinomial log-likelihood is

$$\begin{aligned} & -\sum_{i=1}^{n} \log p_{c_i}(x_i) \\ = & \sum_{k=1}^{K} \sum_{c_i=k} \left[-f_k(x_i) + \ln \left(e^{f_1(x_i)} + \cdots + e^{f_K(x_i)} \right) \right]. \end{aligned}$$

With expression arrays, it is typical that $p \gg n$ (e.g. $n = 35$, $p = 7000$). The consequent

implications on the logistic regression model are:

- The classes are usually linearly separable.
- The log-likelihood achieves a maximum of 0.
- The parameters β_{kj}'s are not uniquely defined, and indeed many will be infinite.

To avoid this problem, we consider the quadratically-regularized negative log-likelihood:

$$H = \sum_{i=1}^{n} \left[-y_i^T f(x_i) + \ln \left(e^{f_1(x_i)} + \cdots + e^{f_K(x_i)} \right) \right] + \frac{\lambda}{2} \sum_{k=1}^{K} \|\beta_k\|^2, \tag{4.5}$$

where y_i is a binary K-vector that re-codes the response c_i, with values all zero except a 1 in position k if the class is k, and

$$f(x_i) = (f_1(x_i), f_2(x_i), \ldots, f_K(x_i))^T .$$

It can be shown that in this quadratically-regularized model, the constraint (4.4) is not necessary anymore, for at the minimum of (4.5), $\sum_{k=1}^{K} \beta_k = 0$ is autumatically satisfied.

In the microarray cancer diagnosis problem, x_{ij} denotes the expression for gene $j = 1, 2, \ldots p$ and sample $i = 1, 2, \ldots n$. Since p is often very large (in the thousands), minimizing (4.5) directly is computationally prohibitive. To reduce the computation, we notice that the score equation of (4.5) is given by:

$$\begin{aligned} \frac{\partial H}{\partial \beta_k} &= \lambda \beta_k - X_{n \times p}^T (y_k - p_k) && (4.6) \\ &= 0, \quad k = 1, \ldots K && (4.7) \end{aligned}$$

where $X_{n \times p}$ is a matrix with x_{ij} as the (i, j)th element, y_k is a $n \times 1$ vector containing $y_{ik}, i = 1, \ldots n$, and p_k is also a $n \times 1$ vector containing $p_k(x_i), i = 1, ldotsn$. The score equation (4.6)-(4.7) indicates that β_k is in the row space of $X_{n \times p}$, or equivalently β_k can be

written as

$$\beta_k = \sum_{i=1}^{n} \alpha_{ik} x_i, \quad k = 1, \dots K. \tag{4.8}$$

Let the Euclidean inner product

$$\langle x, x' \rangle = \sum_{j=1}^{p} x_j x'_j.$$

Then $f_k(x)$ which minimizes H also has the form

$$f_k(x) = b_{k0} + \sum_{i=1}^{n} \alpha_{ik} \langle x, x_i \rangle, \quad a_{ik} \in \mathcal{R} \tag{4.9}$$

and

$$\|\beta_k\|^2 = \sum_{i,i'} \alpha_{ik} \alpha_{i'k} \langle x_i, x_{i'} \rangle. \tag{4.10}$$

Thus (4.5) becomes

$$H = \sum_{i=1}^{n} \left[-y_i^T f(x_i) + \ln \left(e^{f_1(x_i)} + \cdots + e^{f_K(x_i)} \right) \right] + \frac{\lambda}{2} \sum_{k=1}^{K} \sum_{i,i'} \alpha_{ik} \alpha_{i'k} \langle x_i, x_{i'} \rangle. \tag{4.11}$$

The penalized logistic regression model is fit by finding the α_{ik}'s that minimize (4.11), then β_k's can be obtained using (4.8). The number of parameters is reduced from the $(p+1)K$ of (4.5) (pK β_{kj}'s and K β_{k0}'s) to the $(n+1)K$ of (4.11) (nK α_{ik}'s and K β_{k0}'s), making the computations feasible for reasonably small n.

4.2.2 Computational Issues

Since (4.11) is convex, it is natural to use the Newton-Raphson method to minimize H. In order to guarantee convergence, suitable bisection steps can be combined with the Newton-Raphson iterations. The drawback of the Newton-Raphson method is that in each iteration,

an $nK \times nK$ matrix needs to be inverted. Although this can be approximated by a one-step Gauss-Seidel or Jacobian approach (reducing the computation to inverting K $n \times n$ matrices), the computational cost is still high when n or K is large.

Recently Keerthi, Duan, Shevade & Poo (2002) proposed a dual algorithm for two-class penalized logistic regression which avoids inverting huge matrices. It follows the spirit of the popular sequential minimal optimization (SMO) algorithm (Platt (1999)). Preliminary computational experiments show that the algorithm is robust and fast. Keerthi, Duan, Shevade & Poo (2002) describes the algorithm for two-class classification; we have generalized it to the multi-class case. A detailed description of the multi-class case algorithm is given in the Appendix.

4.3 Feature Selection

Besides predicting the correct cancer class for a given tumor sample, another challenge in microarray cancer diagnosis is to identify the relevant genes which contribute most to the classification. In this section, we describe two feature (gene) selection methods.

4.3.1 Univariate Ranking

Several proposals have been made for ranking genes in terms of their classification performance. Golub, Slonim, Tamayo, Huard, Gaasenbeek, Mesirov, Coller, Loh, Downing & Caligiuri (2000) first introduced a ranking criterion for each gene in two-class classification. The criterion is defined as:

$$\rho_j = \left| \frac{\bar{x}_j^{(1)} - \bar{x}_j^{(2)}}{\sigma_j^{(1)} + \sigma_j^{(2)}} \right|,$$

where $\bar{x}_j^{(k)}$ and $\sigma_j^{(k)}$ indicate the average and standard deviation of the gene expression levels of gene j for all samples of class k. Later on Dudoit, Fridlyand & Speed (2002) used the

ratio of between- to within-sum-of-squares of each gene for the multi-class case:

$$\rho_j = \left| \frac{\sum_{k=1}^{K} n_k (\bar{x}_j^{(k)} - \bar{x}_j)^2}{(n-K)\sigma_j^2} \right|, \tag{4.12}$$

where $\bar{x}_j$ is the overall mean expression levels of gene j, n_k is the number of training samples belonging to class k, and σ_j is the pooled within-class standard deviation for gene j:

$$\sigma_j^2 = \frac{1}{n-K} \sum_{k=1}^{K} (n_k - 1)\sigma_j^{(k)2}.$$

The idea is to rank the genes according to the value of ρ_j, and then use the largest m as features in penalized logistic regression model or the support vector machine. m would then be regarded as a nuisance parameter that has to be determined.

Recently, Tibshirani, Hastie, Narasimhan & Chu (2002) proposed the shrunken centroids method for classification. Their soft thresholding approach uses shrinkage and gene selection, integrated into a naive Bayes classifier. It also effectively uses univariate gene selection, based on t-statistics for each gene of each class.

These criteria all implicitly assume orthogonality among the features (genes), because each ρ_j is computed with information about a single gene and does not take into account mutual information between genes.

4.3.2 Recursive Feature Elimination

Guyon, Weston, Barnhill & Vapnik (2002) described another gene selection method based on recursive feature elimination (RFE). At each step of the iterative procedure, a classifier is fitted with all the current features, a ranking criterion is computed for each feature, and the feature with the smallest ranking criterion is removed. A commonly used ranking criterion is defined as:

$$\Delta P_j = \frac{1}{2} \frac{\partial^2 P}{\partial \beta_j^2} \beta_j^2, \tag{4.13}$$

where P is a loss function calculated on the training data, and β_j is the coefficient corresponding to feature j. ΔP_j roughly approximates the sensitivity of P to feature j. For the mean-squared-error classifier ($P = \|y - \beta^T x\|_2^2$) and linear support vector machines, $\Delta P_j = \beta_j^2 \|x_j\|_2^2$ which is similar to a t-statistic, where x_j is the n-vector of sample values for gene j. Assuming that the values of each feature have similar ranges, $\Delta P_j = \beta_j^2$ is also often used. For computational reasons, it may be more efficient to remove a large number of features at a time, at the expense of possibly degrading the classification performance.

4.4 Results

In this section, we fit both penalized logistic regression and the support vector machine models to three cancer diagnosis microarray data sets. We use the two feature selection methods in section 4.3 to reduce the number of genes. For the univariate ranking method (UR), at each step, we use (4.12) and remove the 10% of the genes with the smallest univariate ranking coefficients. For the recursive feature elimination (RFE) with penalized logistic regression, at each step, we fit the model as in (4.2) and (4.9):

$$\begin{aligned} f_k(x) &= \beta_{k0} + \sum_{i=1}^{n} \alpha_{ik} \langle x, x_i \rangle \\ &= \beta_{k0} + \sum_{j=1}^{p} \beta_{kj} x_j \end{aligned}$$

and eliminate the genes with the overall smallest 10% β_{kj}^2 values. So at each step we may eliminate different number of genes for different k. For the support vector machine RFE, since we use the one-vs-rest scheme as described in section 4.1, the β_{kj}^2's are not comparable for different k, hence at each step, we remove the genes with the smallest 10% β_{kj}^2 values for each class k. The number of genes in the final model is selected by cross-validation and the performance of the final model is evaluated on test samples.

Before applying penalized logistic regression, we standardize each sample as is usually done in microarray studies, e.g. Dudoit, Fridlyand & Speed (2002) and Guyon, Weston,

Barnhill & Vapnik (2002), so the mean and standard deviation of the expression levels of each sample are 0 and 1 respectively.

4.4.1 Choosing λ

The regularization paramter λ in (4.5) and (4.11) is chosen by cross-validation without gene selection. The cross-validated deviance is used as the criterion. For example, Figure 4.1 shows the result for the SRBCT data. The SRBCT data is described in more detail in section 4.4.3. The minimum cross-validated deviance corresponds to $\lambda = 1/1024$, hence $\lambda = 1/1024$ is chosen as the regularization parameter. The right panel of Figure 4.1 plots the values of ranking criterion for each gene of each class based on (4.13) for penalized logistic regression model. The chosen $\lambda = 1/1024$ is used and all the genes are in the model. We can see the distribution of the ranking criterion values is highly skewed: most of the genes are clustered in the bottom (with small ranking criterion values), only a few are scattered in the top (with large ranking criterion values). This indicates that most of the genes are redundant genes for the classification. The points with red circles correspond to the genes selected by PLR RFE, which is going to be described in section 4.4.3.

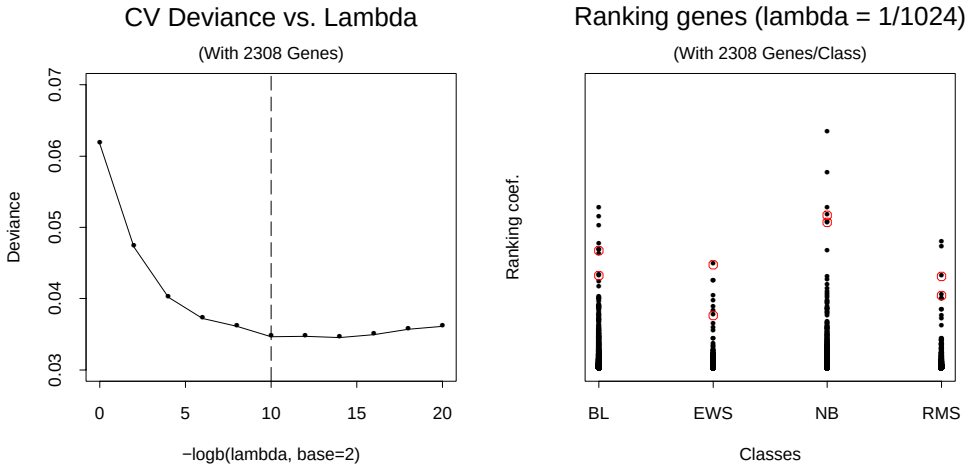

Figure 4.1: *Choosing λ. SRBCT data. Left plot: cross-validated deviance versus λ. All the genes are used. Right plot: ranking criterion values for each gene of each class. The ones with red circles correspond to the genes selected by PLR RFE.*

Table 4.1: Comparison of leukemia classification methods

Method	10-fold CV error	Test error	No. of genes
SVM UR	2/38	3/34	22
PLR UR	2/38	3/34	16
SVM RFE	2/38	1/34	31
PLR RFE	2/38	1/34	26

4.4.2 Leukemia Data

This data set consists of 38 training samples and 34 test samples of two types of acute leukemias, acute myeloid leukemia (AML) and acute lymphoblastic leukemia (ALL) (Golub, Slonim, Tamayo, Huard, Gaasenbeek, Mesirov, Coller, Loh, Downing & Caligiuri (2000)). Each sample is a vector of 7,129 genes. $\lambda = 1/16$ is used.

The results are plotted in Figure 4.2 and summarized in Table 4.1. Notice that for univariate ranking, the rankings of genes only depend on the gene expression levels and do not depend on the fitted model, so penalized logistic regression and the support vector machine use the same set of genes at each step. However in the recursive feature elimination, the β_{kj}^2 values depend on the fitted model, so that at each step, penalized logistic regression and the support vector machine remove different genes. In the upper part of the Figure 4.2, we can see that when using the same set of genes (i.e. using the univariate ranking), penalized logistic regression and the support vector machine give similar classification results, which agrees with the findings of **?**.

The minimum cross-validation error for penalized logistic regression occurs at 135 genes and 60 genes in the univariate ranking and the recursive feature elimination respectively. To obtain a more manageable set of genes, we sacrifice one more cross-validation error (which only increases the cross-validation error from 1 to 2 in both the univariate ranking and the recursive feature elimination) and yields 16 genes and 26 genes. The estimated probabilities from the penalized logistic regression model of the test samples are plotted in the lower part of Figure 4.2. The ones with red circles are the misclassified test samples. Apart from the misclassified one, most other samples have good separation between the two classes (i.e.

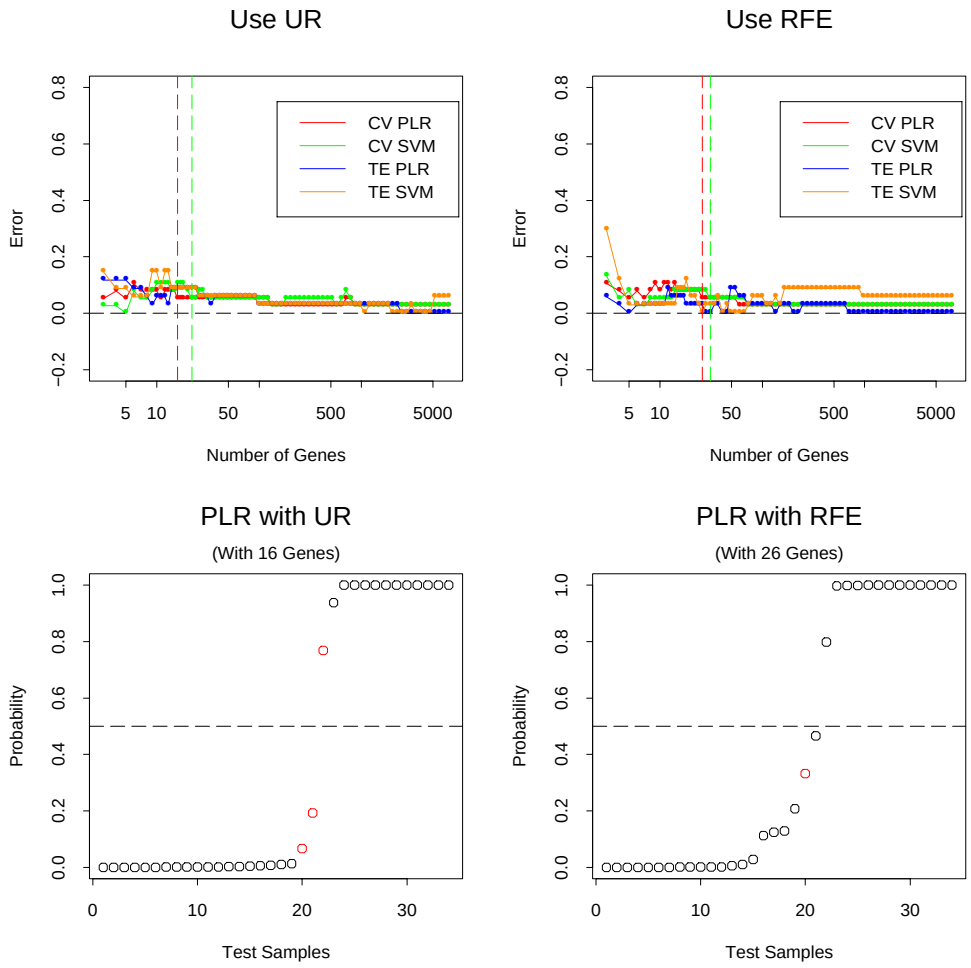

Figure 4.2: *Leukemia classification. Upper plots: cross-validation and test errors of the SVM and the PLR using the UR and the RFE. Lower plots: estimated probabilities for the test data. The ones with red circles are misclassified ones. The horizontal line is the* 0.5 *classification decision boundary.*

the estimated probability is close to either 1 or 0).

4.4.3 SRBCT Data

This data set consists of microarray experiments of small round blue cell tumors (SRBCT) of childhood cancer (Khan, Wei, Ringner, Saal, Ladanyi, Westermann, Berthold, Schwab, Antonescu & Peterson (2001)). The tumors are classified as Burkitt lymphoma (BL), Ewing sarcoma (EWS), neuroblastoma (NB), or rhabdomyosarcoma (RMS). A total of 63 training samples and 25 test samples are provided. Five of the test samples are actually non-SRBCT.

Table 4.2: Comparison of SRBCT classification methods

Method	10-fold CV error	Test error	No. of genes
SVM UR	0/63	0/20	21
PLR UR	0/63	0/20	15
SVM RFE	0/63	0/20	32
PLR RFE	0/63	0/20	8

Each sample consists of expression measurements on 2,308 genes. $\lambda = 1/1024$ is used.

The results are plotted in Figure 4.3 and summarized in Table 4.2. As with the leukemia data, we observe that when using the same set of genes (i.e. using the univariate ranking), penalized logistic regression and the support vector machine give similar classification results, while using the recursive feature elimination, penalized logistic regression tends to reduce more genes than the support vector machine. The penalized logistic regression model choose 14 genes for using the univariate ranking and 8 genes for using the recursive feature elimination. The estimated probabilites from the penalized logistic regression model of the test samples are plotted in the lower part of Figure 4.3. The test error is zero. The ones with violet circles are the maximum estimated probabilites of the five non-SRBCT test samples. Apart from these five samples, most other samples have good separation.

4.4.4 Ramaswamy Data

This data set was described in detail by Ramaswamy, Tamayo, Rifkin, Mukherjee, Yeang, Angelo, Ladd, Reich, Latulippe, Mesirov, Poggio, Gerald, Loda, Lander & Golub (2002). It consists of 144 training tumor samples and 54 test tumor samples, spanning 14 common tumor classes that account for 80% of new cancer diagnoses in the U.S. Among these 54 test samples, 8 are metastatic samples. There are 16,063 genes for each sample. $\lambda = 1/4$ is used.

The results are plotted in Figure 4.4 and Figure 4.5 and summarized in Table 4.3. The prediction results for this data set are not as good as for the other two data sets. This may be due to the relatively large number of cancer classes ($K = 14$), making it harder to

Table 4.3: Comparison of Ramaswamy data classification methods

Method	8-fold CV error	Test error	No. of genes
SVM UR	19/144	12/54	617
PLR UR	20/144	12/54	617
SVM RFE	17/144	9/54	315
PLR RFE	17/144	9/54	294

distinguish between classes.

It is worth noting that all four methods choose a large number of genes when compared to the other two data sets, but the recursive feature elimination method chooses much less genes than the univariate ranking method. The estimated probabilities of the test samples are plotted in Figure 4.5. The ones with black circles are the misclassified test samples and the ones with violet circles are the misclassified metastatic test samples. The maximum probabilites of many of these misclassified samples are not far away from their second highest probabilities. We can also see that the breast, renal and pancreas tumor samples are the most difficult samples for classification.

4.4.5 Discussion

Penalized logistic regression with recursive feature elimination found 26 genes that distinguished acute mylogenous leukemia (AML) from acute lymphocytic leukemia (ALL) with a lower error rate than the methods employed in Golub, Slonim, Tamayo, Huard, Gaasenbeek, Mesirov, Coller, Loh, Downing & Caligiuri (2000) and Tibshirani, Hastie, Narasimhan & Chu (2002). The results of Golub, Slonim, Tamayo, Huard, Gaasenbeek, Mesirov, Coller, Loh, Downing & Caligiuri (2000) and Tibshirani, Hastie, Narasimhan & Chu (2002) are summarized in Table 4.4. Our list of 26 genes includes myeloperoxidase and barely missed terminal deoxynucleotidyl transferase. These two genes were not identified in Golub, Slonim, Tamayo, Huard, Gaasenbeek, Mesirov, Coller, Loh, Downing & Caligiuri (2000) but are known to be excellent markers for AML and ALL, respectively (Tibshirani, Hastie, Narasimhan & Chu (2002)).

Table 4.4: Comparison of leukemia classification methods

Method	10-fold CV error	Test error	No. of genes
Golub *et al.* (1999)	3/38	4/34	50
Tibshirani *et al.* (2002)	2/38	2/34	21
PLR RFE	2/38	1/34	26

Table 4.5: Comparison of SRBCT classification methods

Method	10-fold CV error	Test error	No. of genes
Khan *et al.* (2001)	0/63	0/20	96
Tibshirani *et al.* (2002)	0/63	0/20	43
PLR RFE	0/63	0/20	8

In the SRBCT data set, penalized logistic regression using recursive feature elimination found 2 genes for each class, hence a total of 8 genes were identified. This result seems far superior to that of the neural network method of Khan, Wei, Ringner, Saal, Ladanyi, Westermann, Berthold, Schwab, Antonescu & Peterson (2001), which required 96 genes to obtain zero test error (Table 4.5). Of our 8 genes, 7 were also found by the neural network method, except for "cold shock domain protein A". Comparing our 8 genes to the 43 genes found in Tibshirani, Hastie, Narasimhan & Chu (2002), the 8 gene are all in the list of Tibshirani, Hastie, Narasimhan & Chu (2002). This may imply that the simple linear model used by penalized logistic regression captures the interactions among genes better than the shrunken centroids model.

The Ramaswamy data set has 14 classes. penalized logistic regression using recursive feature elimination identified 294 genes. Given the relatively large number of classes ($K = 14$) and the large number of original genes ($p = 16,063$), this is a sizable reduction. The biological meanings of the genes that were identified need further investigation. Note **?** utilized all $16,063$ genes in their study to achieve an error rate (12/54).

The tendency that penalized logistic regression selects less genes than the support vector machine for the studied data sets may be understood heuristically. For the microarray

cancer diagnosis problems under study, the training sets are usually linearly separable down to just a few genes. Therefore the solution of the support vector machine is rather insensitive to the value of the regularization parameter λ in (4.1). Actually when λ is small enough ($\lambda = 1/128$ is usually adequate), the solution of the support vector machine will be the same for different λ values, which indicates that the regularization does not have much of an effect. This is not the case for penalized logistic regression. Regularization always shrinks the coefficients β_{kj}, no matter what the value of λ is, and makes the irrelevant genes more likely to be removed. This can be also understood from a fundamental difference between the support vector machine and penalized logistic regression. As pointed out in section 4.1, the support vector machine estimates $sign[p(x) - 1/2]$ asymptotically, hence $f(x)$ is close to -1 or 1, and penalized logistic regression estimates the logit of probabilities, hence $f(x)$ is in the range of $-\infty$ and ∞. Therefore, the shrinkage of penalized logistic regression seems to be more effective.

4.5 Summary

We have proposed penalized logistic regression (PLR) for the microarray cancer diagnosis problem. The empirical results on three data sets show that when using the same set of genes, penalized logistic regression and the support vector machine perform similarly in classification, but in addition penalized logistic regression provides an estimate of the underlying probability. When using the recursive feature (gene) elimination method to select relevant genes for cancer classification, penalized logistic regression tends to reduce more genes than the support vector machine.

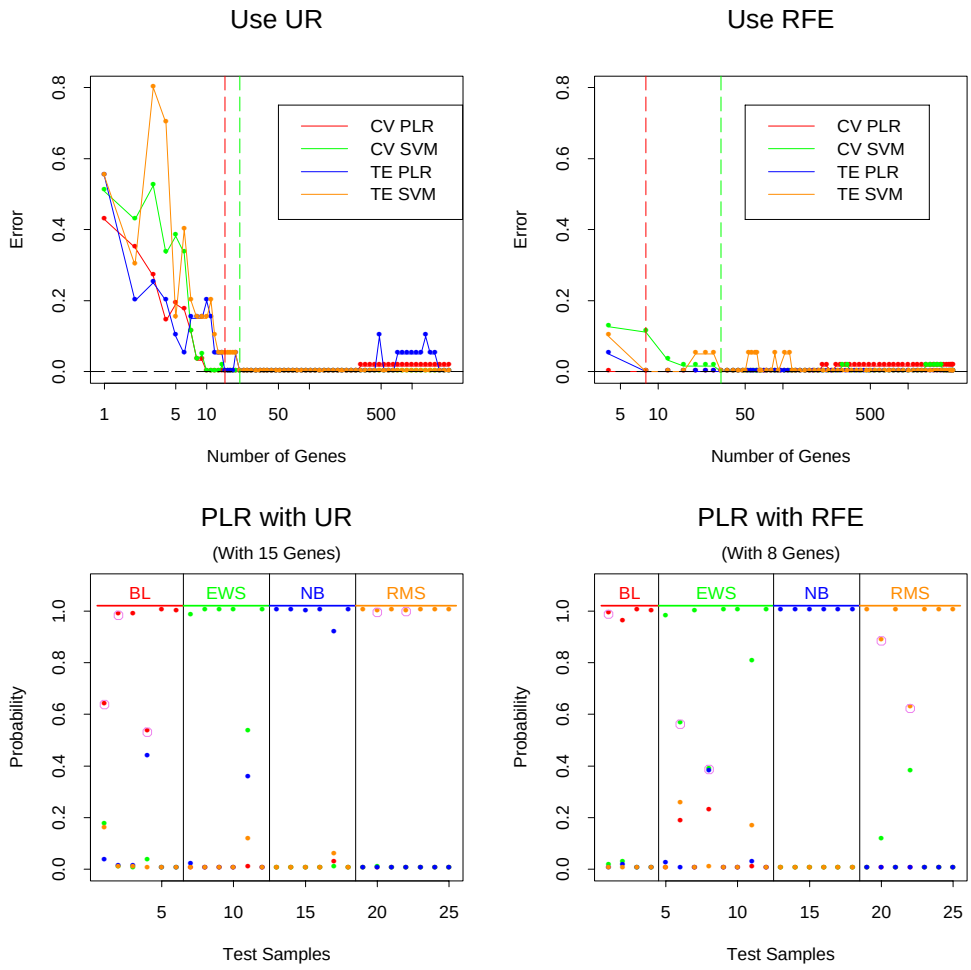

Figure 4.3: *SRBCT classification. Upper plots: cross-validation and test errors of the SVM and the PLR using the UR and the RFE. Lower plots: estimated probabilities for the test data. The samples are partitioned by the predicted class. All* 20 *of the test samples are correctly classified. The* 5 *non-SRBCT test samples are marked with a circle with its maximum estimated probability. They are below the minimum probabilites of the other test samples in each class.*

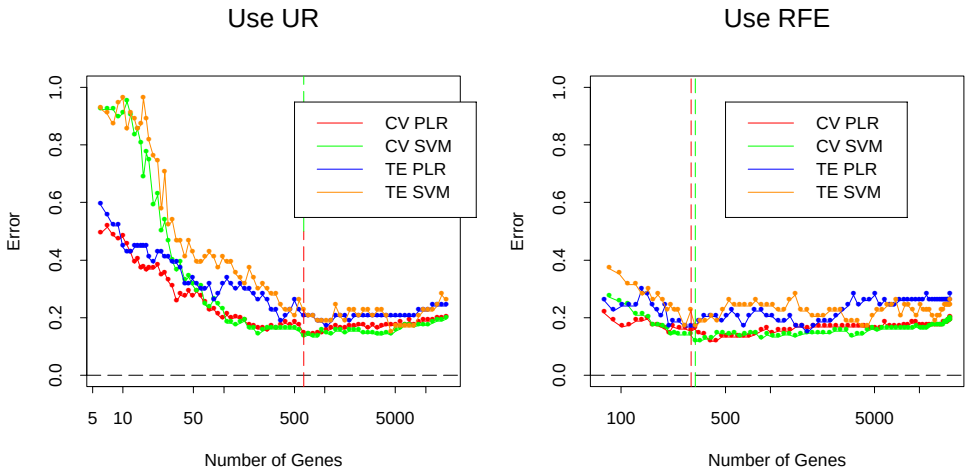

Figure 4.4: *Ramaswamy classification: cross-validation and test errors of the SVM and the PLR using the UR and the RFE.*

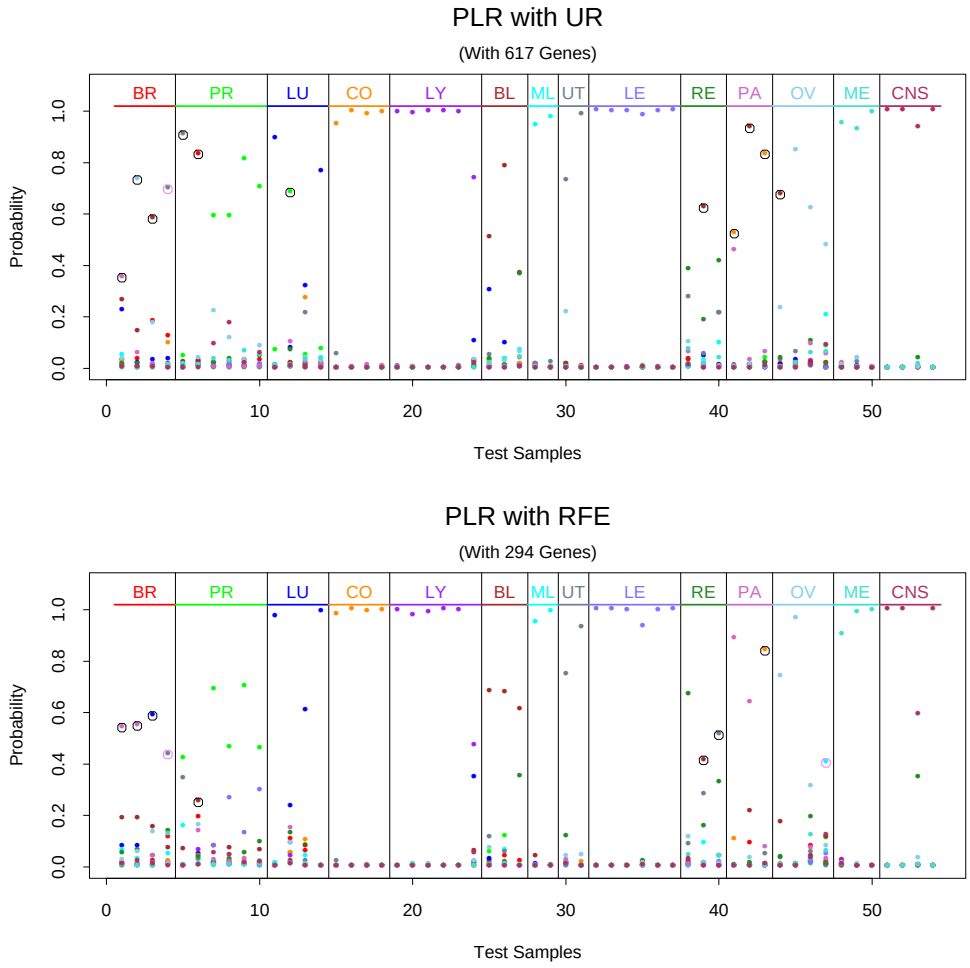

Figure 4.5: *Ramaswamy classification: estimated probabilities for the test data. The samples are partitioned by the true class. The misclassified test samples are marked with black circles with their maximum estimated probabilities and the* 2 *misclassified metastatic test samples are marked with violet circles. The* 14 *classes correspond to: Breast, Prostate, Lung, Colorectal, Lymphoma, Bladder, Melanoma, Ulterus, Leukemia, Renal, Pancreas, Ovary, Mesothelioma and CNS tumors.*

Chapter 5

Summary of Thesis

In Chapter 1, we proposed the import vector machine method for both two-class and multi-class classification problems. We showed that the import vector machine not only performs as well as the support vector machine, but also provides an estimate of the probability $p(x)$. The computational cost of the import vector machine is $O(n^2m^2)$ for the two-class case and $O(Kn^2m^2)$ for the multi-class case, where m is the number of import points.

In Chapter 3, we considered the 1-norm support vector machine. We illustrate that the 1-norm support vector machine may have some advantage over the 2-norm support vector machine, especially when there are redundant features. The solution path $\hat{\beta}(s)$ of the 1-norm support vector machine is a piece-wise linear function in the tuning parameter s. We have proposed an efficient algorithm to compute the whole solution path $\hat{\beta}(s)$ of the 1-norm support vector machine, and facilitate adaptive selection of the tuning parameter s.

In Chapter 4, we concentrate on the microarray classification problem. In particular, we compare the results of penalized logistic regression and the support vector machine. The empirical results on three data sets show that when using the same set of genes, penalized logistic regression and the support vector machine perform similarly in classification, but in addition penalized logistic regression provides an estimate of the underlying probability. When using the recursive feature (gene) elimination method to select relevant genes for cancer classification, penalized logistic regression tends to reduce more genes than the

support vector machine.

Appendix A

Theorems and Proofs

Proof of Theorem 2.1

For the purpose of simple notation, we omit the constant β_0 in the proof. We define

$$f(\beta) \equiv \sum_{i=1}^{n} \ln\left(1 + e^{-y_i h(x_i)^T \beta}\right).$$

Lemma A.1 *Consider the optimizatio problem (2.8)–(2.10), let the solution be denoted by* $\hat{\beta}(s)$. *If the training data are separable, i.e.* $\exists \beta$, *s.t.* $y_i h(x_i)^T \beta > 0$, $\forall i$, *then* $y_i h(x_i)^T \hat{\beta}(s) > 0$, $\forall i$ *and* $\|\hat{\beta}(s)\|_2 = s$ *for all* $s > s_0$, *where* s_0 *is a fixed positive number. Hence,* $\left\| \frac{\hat{\beta}(s)}{s} \right\| = 1$.

Proof

Suppose $\exists i^*$, *s.t.* $y_{i^*} h(x_{i^*})^T \beta \leq 0$, then

$$\begin{aligned} f(\beta) &\geq \ln\left(1 + e^{-y_{i^*} h(x_{i^*})^T \beta}\right) && \text{(A.1)} \\ &\geq \ln 2. && \text{(A.2)} \end{aligned}$$

On the other hand, by the separability assumption, we know there exists $\beta^*,\ \|\beta^*\|_2 = 1$, *s.t.* $y_i h(x_i)^T \beta^* > 0,\ \forall i$. Then for $s > s_0 = -\ln(2^{1/n} - 1)/\min_i \left(y_i h(x_i)^T \beta^*\right)$, we have

$$f(s\beta^*) = \sum_{i=1}^{n} \ln\left(1 + e^{-y_i h(x_i)^T \beta^* s}\right) \tag{A.3}$$

$$< \sum_{i=1}^{n} \frac{\ln 2}{n} = \ln 2. \tag{A.4}$$

Since $f(\hat{\beta}(s)) \leq f(s\beta^*)$, we have, for $s > s_0$, $y_i h(x_i)^T \hat{\beta}(s) > 0,\ \forall i$.

For $s > s_0$, if $\|\hat{\beta}(s)\|_2 < s$, we consider to scale up $\hat{\beta}(s)$ by letting

$$\hat{\beta}'(s) = \frac{\hat{\beta}(s)}{\|\hat{\beta}(s)\|_2} s.$$

Then $\|\hat{\beta}'(s)\|_2 = s$, and

$$f\left(\hat{\beta}'(s)\right) = \sum_{i=1}^{n} \ln\left(1 + e^{-y_i h(x_i)^T \hat{\beta}'(s)}\right) \tag{A.5}$$

$$< \sum_{i=1}^{n} \ln\left(1 + e^{-y_i h(x_i)^T \hat{\beta}(s)}\right) \tag{A.6}$$

$$= f\left(\hat{\beta}(s)\right), \tag{A.7}$$

which is a contradiction. Hence $\|\hat{\beta}(s)\|_2 = s$. □

Now we consider two separating candidates β_1 and β_2, such that $\|\beta_1\|_2 = \|\beta_2\|_2 = 1$. Assume that β_1 separates better, i.e.:

$$m_1 := \min_i y_i h(x_i)^T \beta_1 > m_2 := \min_i y_i h(x_i)^T \beta_2 > 0.$$

Lemma A.2 *There exists some $s_0 = S(m_1, m_2)$ such that $\forall s > s_0$, $s\beta_1$ incurs smaller loss than $s\beta_2$, in other words:*

$$f(s\beta_1) < f(s\beta_2).$$

Proof

Let

$$s_0 = S(m_1, m_2) = \frac{\ln n + \ln 2}{m_1 - m_2},$$

then $\forall s > s_0$, we have

$$\begin{aligned}
\sum_{i=1}^{n} \ln\left(1 + e^{-y_i h(x_i)^T \beta_1 s}\right) &\leq n \ln\left(1 + e^{-s \cdot m_1}\right) && \text{(A.8)} \\
&\leq n \exp(-s \cdot m_1) && \text{(A.9)} \\
&< \frac{1}{2} \exp(-s \cdot m_2) && \text{(A.10)} \\
&\leq \ln\left(1 + e^{-s \cdot m_2}\right) && \text{(A.11)} \\
&\leq \sum_{i=1}^{n} \ln\left(1 + e^{-y_i h(x_i)^T \beta_2 s}\right). && \text{(A.12)}
\end{aligned}$$

The first and the last inequalities imply

$$f(s\beta_1) < f(s\beta_2).$$

□

Given these two lemmas, we can now prove that any convergence point of $\frac{\hat{\beta}(s)}{s}$ must be a margin maximizing separator. Assume β^* is a convergence point of $\frac{\hat{\beta}(s)}{s}$. Denote $m^* := \min_i y_i h(x_i)^T \beta^*$. Since the training data are separable, clearly $m^* > 0$ (since otherwise $f(s\beta^*)$ does not even converge to 0 as $s \to \infty$).

Now, assume some $\tilde{\beta}$ with $\|\tilde{\beta}\|_2 = 1$ has bigger minimal margin $\tilde{m} > m^*$. By continuity of the minimal margin in β, there exists some open neighborhood of β^*:

$$N_{\beta^*} = \{\beta : \|\beta - \beta^*\|_2 < \delta\},$$

and an $\epsilon > 0$, such that:

$$\min_i y_i h(x_i)^T \beta < \tilde{m} - \epsilon, \quad \forall \beta \in N_{\beta^*}.$$

Now by Lemma A.2 we get that there exists some $s_0 = S(\tilde{m}, \tilde{m} - \epsilon)$ such that $s\tilde{\beta}$ incurs smaller loss than $s\beta$ for any $s > s_0,\ \beta \in N_{\beta^*}$. Therefore β^* cannot be a convergence point of $\frac{\hat{\beta}(s)}{s}$.

We conclude that any convergence point of the sequence $\frac{\hat{\beta}(s)}{s}$ must be a margin maximizing separator. If the margin maximizing separator is unique then it is the only possible convergence point, and therefore:

$$\lim_{s\to\infty} \frac{\hat{\beta}(s)}{s} = \arg\max_{\|\beta\|_2=1} \min_i y_i h(x_i)^T \beta.$$

□

In the case that the margin maximizing separating hyper-plane is not unique, this conclusion can easily be generalized to characterize a unique solution by defining tie-breakers: if the minimal margin is the same, then the second minimal margin determines which model separates better, and so on. Only in the case that the whole order statistics of the margins is common to many solutions can there really be more than one convergence point for $\frac{\hat{\beta}(s)}{s}$.

An SMO algorithm for solving multi-class PLR

The spirit of the multi-class SMO algorithm follows that of the two-class SMO algorithm (Keerthi, Duan, Shevade & Poo (2002)), but the details are different.

To minimize (4.5) under (4.3)–(4.4), we can re-write it as:

$$\min \quad P = C\sum_{i=1}^{n} g(\xi_i) + \frac{1}{2}\sum_{k=1}^{K} \|\beta_k\|^2, \tag{A.13}$$

$$\text{subject to} \ : \quad \xi_{ik} = \beta_{k0} + \beta_k^T x_i, \quad \forall i, k, \tag{A.14}$$

$$\sum_{k=1}^{K} \beta_{k0} = 0, \tag{A.15}$$

where $C = 1/\lambda$ is the regularization parameter, and

$$g(\xi_i) = -(y_{i1}\xi_{i1} + \ldots + y_{iK}\xi_{iK}) + \ln\left(e^{\xi_{i1}} + \ldots + e^{\xi_{iK}}\right).$$

The Lagrangian for this problem is:

$$L = C\sum_{i=1}^{n} g(\xi_i) + \frac{1}{2}\sum_{k=1}^{K}\|\beta_k\|^2 + \sum_{k=1}^{K}\sum_{i=1}^{n}\alpha_{ik}[\xi_{ik} - \beta_{k0} - \beta_k^T x_i] + \alpha_0\sum_{k=1}^{K}\beta_{k0}, \tag{A.16}$$

where α_{ik}'s and α_0 are the Lagrangian multipliers. Therefore the KKT optimality conditions are given by:

$$\frac{\partial L}{\partial \beta_{k0}} = \alpha_0 - \sum_{i=1}^{n}\alpha_{ik} = 0, \quad \forall k, \tag{A.17}$$

$$\nabla_{\beta_k} L = \beta_k - \sum_{i=1}^{n}\alpha_{ik}x_i = 0, \quad \forall k, \tag{A.18}$$

$$\frac{\partial L}{\partial \xi_{ik}} = C\left(-y_{ik} + \frac{e^{\xi_{ik}}}{\sum_{k'=1}^{K} e^{\xi_{ik'}}}\right) + \alpha_{ik} = 0, \quad \forall i, k, \tag{A.19}$$

which allow us to express β_k and ξ_{ik} as functions of α_{ik}'s:

$$\beta_k = \sum_{i=1}^{n}\alpha_{ik}\vec{x}_i, \quad \forall k, \tag{A.20}$$

$$\xi_{ik} = \ln\left(y_{ik} - \frac{\alpha_{ik}}{C}\right) - \frac{1}{K}\sum_{k'=1}^{K}\ln\left(y_{ik'} - \frac{\alpha_{ik'}}{C}\right), \quad \forall i, k. \tag{A.21}$$

Here we enforce $\sum_{k=1}^{K}\xi_{ik} = 0, \ \forall i$. Note (A.19) also indicates:

$$\sum_{k=1}^{K}\alpha_{ik} = 0, \quad \forall i. \tag{A.22}$$

Hence we have $\alpha_0 = 0$ and $\sum_{i=1}^{n}\alpha_{ik} = 0, \ \forall k$.

Now we apply Wolfe duality theory to (A.13). The Wolfe dual corresponds to the maximization of L subject to (A.17), (A.18) and (A.19). Using (A.20) and (A.21) we can

simplify the Wolfe dual as:

$$\min \qquad D = C\sum_{i=1}^{n} G(\frac{\alpha_i}{C}) + \frac{1}{2}\sum_{k=1}^{K}\|\beta_k\|^2, \tag{A.23}$$

$$\text{subject to} \quad : \quad \sum_{k=1}^{K}\alpha_{ik} = 0, \quad \forall i, \tag{A.24}$$

$$\sum_{i=1}^{n}\alpha_{ik} = 0, \quad \forall k, \tag{A.25}$$

where $\alpha_i = (\alpha_{i1}, \ldots, \alpha_{iK})^T$, β_k is given by (A.20), and

$$G(\frac{\alpha_i}{C}) = \sum_{k=1}^{K}(y_{ik} - \frac{\alpha_{ik}}{C})\ln(y_{ik} - \frac{\alpha_{ik}}{C}).$$

Once the dual problem is solved for α_{ik}'s, the primal variables β_k's and ξ_{ik}'s can be obtained using (A.20) and (A.21).

To solve (A.23), we first replace α_{iK} with $-\sum_{k=1}^{K-1}\alpha_{ik}$, hence (A.23) becomes

$$\min \qquad D = C\sum_{i=1}^{n} G(\frac{\alpha_i}{C}) + \frac{1}{2}\sum_{k=1}^{K-1}\|\beta_k\|^2 + \frac{1}{2}\|\beta_K\|^2, \tag{A.26}$$

$$\text{subject to} \quad : \quad \sum_{i=1}^{n}\alpha_{ik} = 0, \quad k = 1, \ldots K-1, \tag{A.27}$$

where $\beta_K = -\sum_{i=1}^{n}\left(\sum_{k=1}^{K-1}\alpha_{ik}\right)x_i$ and

$$G(\frac{\alpha_i}{C}) = \sum_{k=1}^{K-1}(y_{ik} - \frac{\alpha_{ik}}{C})\ln(y_{ik} - \frac{\alpha_{ik}}{C}) \tag{A.28}$$

$$+ \left[1 - \sum_{k=1}^{K-1}(y_{ik} - \frac{\alpha_{ik}}{C})\right] \cdot \ln\left[1 - \sum_{k=1}^{K-1}(y_{ik} - \frac{\alpha_{ik}}{C})\right]. \tag{A.29}$$

Now we can write down the optimality conditions for (A.26). The Lagrangian for (A.26)

is then

$$\tilde{L} = C\sum_{i=1}^{n} G(\frac{\alpha_i}{C}) + \frac{1}{2}\sum_{k=1}^{K-1}\|\beta_k\|^2 + \frac{1}{2}\|\beta_K\|^2 - \sum_{k=1}^{K}[b_k\sum_{i=1}^{n}\alpha_{ik}].$$

Define

$$\begin{aligned} F_{ik} &\equiv \beta_k^T\cdot x_i - \beta_K^T\cdot x_i + C\frac{\partial G}{\partial \alpha_{ik}} \\ &= (\beta_k-\beta_K)^T\cdot x_i - \left(\ln(y_{ik}-\frac{\alpha_{ik}}{C}) - \ln\left[1-\sum_{k=1}^{K-1}(y_{ik}-\frac{\alpha_{ik}}{C})\right]\right). \end{aligned}$$

Then the KKT conditions for (A.26) are:

$$\frac{\partial\tilde{L}}{\partial\alpha_{ik}} = F_{ik} - b_k = 0 \quad i = 1,\ldots n; k = 1,\ldots K-1. \tag{A.30}$$

Define

$$\begin{aligned} i\_up(k) &= \text{argmax}_i F_{ik}, \quad k = 1,\ldots K-1, \\ i\_low(k) &= \text{argmin}_i F_{ik}, \quad k = 1,\ldots K-1. \end{aligned}$$

Then the optimaliy conditions will hold at given α_{ik}'s if and only if

$$F_{i\_up(k),k} = F_{i\_low(k),k} \quad k = 1,\ldots K-1. \tag{A.31}$$

Note to make the F_{ik}'s appropriately defined, the α_{ik}'s must satisfy:

$$\begin{cases} 0 < \alpha_{ik} < C & \text{if } y_{ik} = 1 \\ -C < \alpha_{ik} < 0 & \text{if } y_{ik} = 0, \end{cases} \tag{A.32}$$

and

$$0 < \sum_{k=1}^{K-1}(y_{ik} - \frac{\alpha_{ik}}{C}) < 1 \tag{A.33}$$

If there exists an index pair (i, i') such that

$$F_{ik} \neq F_{i'k} \tag{A.34}$$

for some k, then we can decrease D (A.26) (while maintaining the equality constraint $\sum_{i=1}^{n} \alpha_{ik} = 0$ (A.51)) by adjusting α_{ik} and $\alpha_{i'k}$ only. To see this, we define

$$\begin{aligned} \tilde{\alpha}_{ik}(t) &= \alpha_{ik} + t, \\ \tilde{\alpha}_{i'k}(t) &= \alpha_{i'k} - t, \\ \tilde{\alpha}_{i'k''}(t) &= \alpha_{i'k''} \quad \text{for all others.} \end{aligned}$$

Then one can verify that

$$\frac{dD}{dt} = F_{ik} - F_{i'k},$$

where F_{ik} and $F_{i'k}$ are evaluated at t. Since by (A.34), $F_{ik} - F_{i'k} \neq 0$ at $t = 0$, a decrease in D is possible by choosing t suitably away from 0.

The SMO algorithm can now be described as follows:

$(B1)$ Choose α^0 satisfying conditions (A.51) and (A.52). Set $r = 1$.

$(B2)$ If α^r satisfies (A.31), stop. If not, let

$$\alpha_{i\_low,k^*} = \alpha^r_{i\_low,k^*} + t, \tag{A.35}$$

$$\alpha_{i\_up,k^*} = \alpha^r_{i\_up,k^*} - t, \tag{A.36}$$

$$\alpha_{i,k} = \alpha^r_{i,k} \quad \text{for other} \quad i, k. \tag{A.37}$$

Find t^* which minimizes the dual D (A.26).

$(B3)$ Update α^{r+1} according to (A.35), (A.36) and (A.37). Go back to step $(B2)$.

Notice that the step $(B2)$ is a univariate optimization problem, and updating formulas can be used to calculate F_{ik}'s and D.

Proof of Theorem A.1

Theorem A.1 *If the solution path of (3.1)–(3.2) is unique, then the algorithm described in section 3.3.2 – 3.3.3 indeed gives the exact whole solution path* $\hat{\beta}(s)$.

Proof

Recall the notation we used in section 3.3:

$$\mathcal{E} = \{i : 1 - y_i f_i = 0\}, \text{for points at the } \textit{elbow} \text{ of the loss function;} \tag{A.38}$$

$$\mathcal{L} = \{i : 1 - y_i f_i > 0\}, \text{for points to the } \textit{left} \text{ of the elbow;} \tag{A.39}$$

$$\mathcal{R} = \{i : 1 - y_i f_i < 0\}, \text{for points to the } \textit{right} \text{ of the elbow;} \tag{A.40}$$

$$\mathcal{V} = \{j : \hat{\beta}_j \neq 0\}, \text{for basis functions with non-zero coefficients.} \tag{A.41}$$

Consider an equivalent setup of (3.1)–(3.2):

$$\min_{\beta_0, \beta} \quad \sum_{i=1}^{n} \epsilon_i \tag{A.42}$$

$$\text{s.t.} \quad 1 - y_i f_i \leq \epsilon_i, \ \epsilon_i \geq 0 \tag{A.43}$$

$$f_i = \beta_0 + \sum_{j=1}^{q} \beta_j h_j(x_i) \tag{A.44}$$

$$\sum_{j=1}^{q} |\beta_j| \leq s. \tag{A.45}$$

The Lagrange dual of (A.42)–(A.45) is:

$$L = \sum_{i=1}^{n} \epsilon_i + \lambda \sum_{j=1}^{q} |\beta_j| + \sum_{i=1}^{n} \alpha_i (1 - y_i f_i - \epsilon_i) - \sum_{i=1}^{n} \gamma_i \epsilon_i,$$

where λ, α_i and γ_i are non-negative Lagrange multipliers. The corresponding Karush-Kuhn-Tucker conditions are:

$$sign(\beta_j)\lambda - \sum_i \alpha_i y_i h_j(x_i) = 0 \text{ for } j \in \mathcal{V} \quad (A.46)$$

$$\sum_i \alpha_i y_i = 0 \quad (A.47)$$

$$1 - \alpha_i - \gamma_i = 0 \quad (A.48)$$

$$\alpha_i(1 - y_i f_i - \epsilon_i) = 0 \quad (A.49)$$

$$\gamma_i \epsilon_i = 0 \quad (A.50)$$

Since we are interested in the whole solution path $\hat{\beta}(s)$, we care what happens when the tuning parameter s changes. As s increases, $\hat{\beta}$ and f will change. We define an *event* to be either a point hits the elbow, i.e. a residual $1 - y_i f_i$ changes from non-zero to zero, or a coefficient $\hat{\beta}_j$ changes from non-zero to zero, when s increases. The Karush-Kuhn-Tucker conditions imply that:

- $1 - y_i f_i > 0 \Rightarrow \epsilon_i > 0 \Rightarrow \gamma_i = 0 \Rightarrow \alpha_i = 1$.
- $1 - y_i f_i < 0 \Rightarrow \epsilon_i = 0 \Rightarrow (1 - y_i f_i - \epsilon_i) > 0 \Rightarrow \alpha_i = 0$.
- $1 - y_i f_i = 0 \Rightarrow \epsilon_i = 0 \Rightarrow \gamma_i$ free $\Rightarrow 0 \leq \alpha_i \leq 1$.

Hence we can see:

- When s is fixed, for the solution to be unique, we have $|\mathcal{V}| = |\mathcal{E}|$, and the set of equations that α_i's satisfy is:

$$sign(\hat{\beta}_j)\lambda - \sum_{\mathcal{E}} y_i h_j(x_i)\alpha_i = \sum_{\mathcal{L}} y_i h_j(x_i), \text{ for } j \in \mathcal{V} \quad (A.51)$$

$$\sum_{\mathcal{E}} y_i \alpha_i = -\sum_{\mathcal{L}} y_i. \quad (A.52)$$

  When s increases, for continuity reasons, $\mathcal{V}$, $\mathcal{L}$ and $\mathcal{R}$ will not change, unless an *event* happens. Hence for the solution to exist and be unique, the set of equations that

α_i's satisfy will not change, and correspondingly α_i's will not change, unless an *event* happens.

- When α_i's do not change, the points at the elbow, i.e. points in $\mathcal{E}$, will stay at the elbow. Therefore, $\hat{\beta}_j$'s satisfy:

$$1 - y_i(\hat{\beta}_0 + \sum_{\mathcal{V}} \hat{\beta}_j h_j(x_i)) = 0 \text{ for } i \in \mathcal{E}. \tag{A.53}$$

  Since $|\mathcal{V}| = |\mathcal{E}|$, there is one free unknown variable in this set of equations. Hence we conclude:

- Between two events, $\hat{\beta}(s)$ changes linearly when s increases. To compute the whole solution path $\hat{\beta}(s)$, all we need to do is to find the *joints*, i.e. the asterisk points in Figure 3.1, on this piece-wise linear path, then use straight lines to interpolate them, or equivalently, to start at $\hat{\beta}(0) = 0$, find the right derivative of $\hat{\beta}(s)$, let s increase and only change the derivative when $\hat{\beta}(s)$ gets to a joint.

Now we show that the algorithm described in section 3.3.2 – section 3.3.3 agree with the implication of the Karush-Kuhn-Tucker conditions.

Initial solution (i.e. s=0)

Without loss of generality, we assume $\#\{y_i = 1\} \geq \#\{y_i = -1\}$; then $\hat{\beta}_0(0) = 1, \hat{\beta}_j(0) = 0$, i.e. all the negative points are in $\mathcal{L}$ and all the positive points are in $\mathcal{E}$. When s increases away from 0, some positive points will leave the elbow and others will stay at the elbow; for continuity reason, the negative points will still be in $\mathcal{L}$ if s is small enough. That means the α_i's will not change until some negative point hits the elbow when s increases, and before this negative point hits the elbow, $\hat{\beta}(s)$ changes linearly in a way such that the points in $\mathcal{E}$ stay at the elbow. To get the right derivative of $\hat{\beta}(s)$ at 0, we can solve (A.42) – (A.45) for a small enough s and compute $\hat{\beta}(s) - \hat{\beta}(0)/s$, but we do not know in advance how small s should be. To get around this difficulty, we consider the modified problem (3.9) – (3.10). Notice that if $y_i = 1$, the loss is still $(1 - y_i f_i)_+$; but if $y_i = -1$, the loss becomes $(1 - y_i f_i)$.

Hence even when a negative point hits the elbow, i.e. $1 - y_i f_i = 0$, since 0 is not a non-smooth point of $(1 - y_i f_i)$ anymore, the α_i's will not change. Therefore, the right derivative of $\hat{\beta}(\Delta s)$ with respect to Δs is the same no matter what value Δs is, and it coincides with the right derivative of $\hat{\beta}(s)$ when s is sufficiently small.

Main algorithm

When s increases, or equivalently Δs increases in (3.11), the Karush-Kuhn-Tucker conditions (A.46)–(A.50) will continue to hold and the points in $\mathcal{E}$ will stay at the elbow, until an *event* happens. By the definition of *event*, when an event happens, there will be $|\mathcal{V}| + 1$ points at the elbow and $|\mathcal{V}|$ basis functions with non-zero coefficients. Hence to keep the Karush-Kuhn-Tucker conditions hold, we need to either add a basis function not in $\mathcal{V}$ into $\mathcal{V}$, or remove a point in $\mathcal{E}$ from $\mathcal{E}$.

Step 2 and step 3 of section 3.3.3 exhaust these possibilities. The first equations of (3.12) and (3.14) make sure $\hat{\beta}(s)$ changes in a way such that the points at the elbow will stay at the elbow. The second equations of (3.12) and (3.14) guarantee that $|\hat{\beta}(s)|$ increases when Δs increases. Step 4 makes the choice that decreases the *loss* with the fastest rate among all possibilities.

Now we show that $-\frac{\Delta loss}{\Delta s} = \lambda$. From (A.46), we have

$$sign(\beta_j)\lambda = \sum_i \alpha_i y_i h_j(x_i) \tag{A.54}$$

$$= \sum_{\mathcal{L}} y_i h_j(x_i) + \sum_{\mathcal{E}} \alpha_i y_i h_j(x_i), \quad \text{for } j \in \mathcal{V}. \tag{A.55}$$

Multiply each equation with u_j (u is from (3.20)) and add them together, we have

$$\begin{aligned}
\lambda \sum_{\mathcal{V}} sign(\beta_j) u_j &= \sum_{\mathcal{L}} \sum_{\mathcal{V}} y_i u_j h_j(x_i) + \sum_{\mathcal{E}} \sum_{\mathcal{V}} \alpha_i y_i u_j h_j(x_i) && \text{(A.56)} \\
&= \sum_{\mathcal{L}} y_i \sum_{\mathcal{V}} u_j h_j(x_i) + \sum_{\mathcal{L}} y_i u_0 + \sum_{\mathcal{E}} \alpha_i y_i \sum_{\mathcal{V}} u_j h_j(x_i) \\
&\quad + \sum_{\mathcal{E}} \alpha_i y_i u_0 - \sum_{\mathcal{L}} y_i u_0 - \sum_{\mathcal{E}} \alpha_i y_i u_0 && \text{(A.57)} \\
&= \sum_{\mathcal{L}} y_i (u_0 + \sum_{\mathcal{V}} u_j h_j(x_i)) + \sum_{\mathcal{E}} \alpha_i y_i (u_0 + \sum_{\mathcal{V}} u_j h_j(x_i)) && \text{(A.58)} \\
&= \sum_{\mathcal{L}} y_i (u_0 + \sum_{\mathcal{V}} u_j h_j(x_i)) && \text{(A.59)} \\
&= -\frac{\Delta loss}{\Delta s}. && \text{(A.60)}
\end{aligned}$$

In (A.58), we used (A.47) and the fact that $\alpha_i = 1,$ for $i \in \mathcal{L}$ and $\alpha_i = 0,$ for $i \in \mathcal{R}$. In (A.59), we used the fact that the points in $\mathcal{E}$ stay at the elbow, hence $u_i + \sum_{\mathcal{V}} u_j h_j(x_i) = 0,$ for $i \in \mathcal{E}$. Since $\sum_{\mathcal{V}} sign(\beta_j) u_j = 1$, the left hand side of (A.56) is equal to λ. Therefore we conclude

$$\lambda = -\frac{\Delta loss}{\Delta s}.$$

Since we know as s increases, λ will decrease, this result justifies that the choice made in step 4 of section 3.3.3 agrees with the Karush-Kuhn-Tucker conditions. It also implies that unlike s, λ changes discontinuously.

To be more rigorous, it remains for the algorithm to check that the α_i's computed from (A.51)–(A.52) satisfy: $0 \leq \alpha_i \leq 1$. To simply the computation, we omit this step in section 3.3.3. Our experiments indicate that it does not make a difference.

Thus, we have shown that section 3.3.2 – section3.3.3 computes a path $\hat{\beta}(s)$ that satisfies the Karush-Kuhn-Tucker conditions at any point on the path. By the Karush-Kuhn-Tucker theorem, if the solution path to (A.42)–(A.45) is unique, then $\hat{\beta}(s)$ is the solution path.

□

Bibliography

Bradley, P. & Mangasarian, O. (1998), Feature selection via concave minimization and support vector machines, *in* J. Shavlik, ed., 'ICML'98', Morgan Kaufmann.

Burges, C. (1998), 'A tutorial on support vector machines for pattern recognition', *Data Mining and Knowledge Discovery* **2(2)**, 121–167.

Dudoit, S., Fridlyand, J. & Speed, T. (2002), 'Comparison of discrimination methods for the classification of tumors using gene expression data', *Journal of the American Statistical Association* **97**, 77–87.

Evgeniou, T., Pontil, M. & Poggio, T. (1999), Regularization networks and support vector machines, *in* A. Smola, P. Bartlett, B. Schölkopf & C. Schuurmans, eds, 'Advances in Large Margin Classifiers', MIT Press.

Friedman, J., Hastie, T., Rosset, S., Tibshirani, R. & Zhu, J. (2004), 'Discussion of "consistency in boosting" by w. jiang, g. lugosi, n. vayatis and t. zhang', *Annals of Statistics* p. To appear.

Golub, T., Slonim, D., Tamayo, P., Huard, C., Gaasenbeek, M., Mesirov, J., Coller, H., Loh, M., Downing, J. & Caligiuri, M. (2000), 'Molecular classification of cancer: class discovery and class prediction by gene expression monitoring', *Science* **286**, 531–536.

Green, P. & Yandell, B. (1985), Semi-parametric generalized linear models, *in* 'Proceedings 2nd International GLIM Conference', Vol. 32 of *Lecture notes in Statistics*, Springer-Verlag, New York, pp. 44–55.

Guyon, I., Weston, J., Barnhill, S. & Vapnik, V. (2002), 'Gene selection for cancer classification using support vector machines', *Machine Learning* **46**, 389–422.

Hastie, T. & Tibshirani, R. (1990), *Generalized Additive Models*, Series in Applied Mathematics, Chapman and Hall.

Hastie, T., Tibshirani, R. & Friedman, J. (2001), *The Elements of Statistical Learning*, Springer-Verlag, New York.

Keerthi, S., Duan, K., Shevade, S. & Poo, A. (2002), 'A fast dual algorithm for kernel logistic regression', *International Conference on Machine Learning* **19**.

Khan, J., Wei, J., Ringner, M., Saal, L., Ladanyi, M., Westermann, F., Berthold, F., Schwab, M., Antonescu, C. & Peterson, C. (2001), 'Classification and diagnostic prediction of cancers using gene expression profiling and artificial neural networks', *Nature Medicine* **7**, 673–679.

Kimeldorf, G. & Wahba, G. (1971), 'Some results on tchebycheffian spline functions', *J. Math. Anal. Applic.* **33**, 82–95.

Lee, Y. & Lee, C. (2003), 'Classification of multiple cancer types by multicategory support vector machines using gene expression data', *Bioinformatics* **19**, 1132–1139.

Lee, Y., Lin, Y. & Wahba, G. (2002), 'Multicategory support vector machines, theory, and application to the classification of microarray data and satellite radiance data', *Journal of the American Statistical Association* .

Lin, X., Wahba, G., Xiang, D., Gao, F., Klein, R. & Klein, B. (2000), 'Smoothing spline anova models for large data sets with bernoulli observations and the randomized gacv', *Annals of Statistics* **28**, 1570–1600.

Lin, Y. (2002), 'Support vector machine and the bayes rule in classification', *Data Mining and Knowledge Discovery* **6**, 259–275.

Mukherjee, S., Tamayo, P., Slonim, D., Verri, A., Golub, T., Mesirov, J. & Poggio, T. (1999), Support vector machine classification of microarray data, Technical report, AI Memo 1677, MIT.

Platt, J. (1999), Fast training of support vector machines using sequential minimal optimization, *in* B. Schölkopf, C. Burges & A. Smola, eds, 'Advances in Kernel Methods - Support Vector Learning', MIT Press.

Ramaswamy, S., Tamayo, P., Rifkin, R., Mukherjee, S., Yeang, C., Angelo, M., Ladd, C., Reich, M., Latulippe, E., Mesirov, J., Poggio, T., Gerald, W., Loda, M., Lander, E. & Golub, T. (2002), 'Multiclass cancer diagnosis using tumor gene expression signature', *PNAS* **98**, 15149–15154.

Rätsch, G., T.Onoda & K.R.Müller (2000), 'Soft margins for adaboost', *Machine Learning* pp. 1–35.

Rosset, S., Zhu, J. & Hastie, T. (2003), Boosting as a regularized path to a maximum margin classifier, Technical report, Department of Statistics, Stanford University.

Smola, A. & Schölkopf, B. (2000), Sparse greedy matrix approximation for machine learning, *in* 'Proceedings of the Seventeenth International Conference on Machine Learning', Morgan Kaufmann.

Song, M., Breneman, C., Bi, J., Sukumar, N., Bennett, K., Cramer, S. & Tugcu, N. (2002), 'Prediction of protein retention times in anion-exchange chromatography systems using support vector regression', *Journal of Chemical Information and Computer Sciences* p. September.

Tibshirani, R. (1996), 'Regression shrinkage and selection via the lasso', *Journal of the Royal Statistical Society Series B* **58**, 267–288.

Tibshirani, R., Hastie, T., Narasimhan, B. & Chu, G. (2002), 'Diagnosis of multiple cancer types by shrunken centroids of gene expression', *PNAS* **99**, 6567–6572.

Vapnik, V. (1995), *The Nature of Statistical Learning Theory*, Springer-Verlag, New York.

Vapnik, V. (1998), *Statistical Learning Theory*, Wiley, New York.

Wahba, G. (1990), *Spline Models for Observational Data*, Vol. 59 of *Series in Applied Mathematics*, SIAM, Philadelphia.

Wahba, G. (1999), Support vector machines, reproducing kernel hilbert space and the randomized gacv, *in* B. Schölkopf, C. Burges & A. Smola, eds, 'Advances in Kernel Methods - Support Vector Learning', MIT Press, pp. 69–88.

Wahba, G., Gu, C., Wang, Y. & Chappell, R. (1995), Soft classification, a.k.a. risk estimation, via penalized log likelihood and smoothing spline analysis of variance, *in* D. Wolpert, ed., 'The Mathematics of Generalization', Santa Fe Institute Studies in the Sciences of Complexity. Addison-Wesley Publisher.

Weston, J. & Watkins, C. (1999), Multi-class support vector machines, *in* M. Verleysen, ed., 'Proceedings of ESANN99', D. Facto Press, Brussels.

Williams, C. & Seeger, M. (2001), Using the nystrom method to speed up kernel machines, *in* T. Leen, T. Diettrich & V. Tresp, eds, 'Advances in Neural Information Processing Systems', Vol. 13, MIT Press.